Машинное обучение
Лекция 1
Введение. Основные задачи машинного обучения

Место в математике и информатике

Машинное обучение (Machine Learning) (МО) (компьютерное обучение) –

раздел искусственного интеллекта (интеллектуальных систем), занимающийся построением моделей на основе набора известных данных. Относится к компьютерным наукам (Computer Sciences).

Связь МО и разделов математики и информатики:

- математическая статистика;
- математическое моделирование;
- линейная алгебра;
- дискретная математика;
- нейронные сети;
- нечёткая логика;
- методы оптимизации.

1. Основные задачи машинного обучения

1.1. Понятие МО I

Машинное обучение (МО) –

разработка и исследование моделей и алгоритмов выявления закономерностей (знаний) на основе обучения по множеству имеющихся примеров — конкретных известных данных.

МО (Артур Самуэль, 1959 г.) –

«процесс, в результате которого машина (компьютер) способна показывать поведение, которое в нее не было явно заложено (запрограммировано)».

(Том Митчел) –

«Компьютерная программа обучается на основе опыта E по отношению к некоторому классу задач T и меры качества P, если качество решения задач из T, измеренное на основе P, улучшается с приобретением опыта E.»

Основная цель МО – выявление **скрытых полезных зависимостей, моделей** в массивах информации (для прогнозирования поведения системы в будущем и пр. задач).
МО – поиск структуры в известных данных.

Мнение: МО ≡ Data Mining (извлечение знаний из данных). Важно повышение достоверности обучения с увеличением объемов данных ⇒, МО связано с Big Data (большими данными).

[Ian H. Witten and others]: МО – техническая основа Data Mining.

Инструменты машинного обучения: Python, R.

1.2. Виды обучения I

Классы обучения:

1. дедуктивное;
2. индуктивное.

Дедуктивное обучение –

формализованное представление знаний, известных экспертам, как правило, в виде одной из моделей представления знаний (логическая, продукционная, фреймовая модель, семантическая сеть.). Этот вид обучения относится к разделам **инженерии знаний и экспертных систем**.

Индуктивное обучение

опирается на множество прецедентов – известной информации об исследуемой предметной области. Также называется обучением по прецедентам.

Типы обучения:

1. с учителем (supervised);
2. без учителя (unsupervised);
3. с подкреплением (reinforcement);
4. частичное (semi-supervised).

Особенность МО – необходимость уточнения моделей в процессе увеличения объёмов известных данных. В связи с этим, в задачи МО включены методы подготовки и предварительной обработки исходных множеств данных, если они обладают большой размерностью:

- задачи снижения размерности пространства переменных;
- снижение объема данных;
- выявление противоречивых данных;
- заполнение пропусков в данных;
- адаптация результатов к новым данным;

- проверка достоверности полученных зависимостей.

Актуальность разработки методов МО выросла в связи со **значительным ростом технических возможностей хранения и обработки огромных объёмов данных**.

Предположение МО –

в данных имеется некая зависимость, присутствует какое-то знание. Методы МО нацелены на выделение определенного вида зависимостей. Дать ответ на вопрос, имеется ли полезная информация в данных, **не представляется возможным**.

1.3. Примеры задач МО I

Некоторые задачи, решаемые МО:

- распознавание образов;
- диагностика;
- обнаружение спама;
- прогнозирование финансовых показателей;
- анализ социальных сетей;
- обнаружение мошенничества;
- кредитный скоринг,;
- оценка релевантности ресурсов в информационном поиск;
- рекомендательные системы.

Высок спрос на приложение МО к решению задач в области бизнеса (цель – повышение уровня продаж товаров или предоставления услуг за счёт анализа поведения потребителей).

1.4. Классификаций шкал измерений I

Шкала –

числовая система, в которой отношения между различными свойствами изучаемых явлений, процессов переведены в свойства того или иного множества, как правило – множества чисел.

Типы шкал (1):

- дискретные шкалы;
- непрерывные шкалы.

Типы шкал (2):

- шкалы отношений: можно узнать, во сколько раз объект больше (меньше);
- интервальные шкалы (допустимы линейные преобразования, нет естественного начала отсчета, как в шкале температур);
- порядковые (ранговые) шкалы: имеет значение порядок следования переменной (шкала рангов, шкала баллов);
- номинальные шкалы (шкалы наименований): отношения «равно» и «не равно» (бинарная переменная, характеристика).

Вопрос:

какая шкала наиболее мощная, какая – наиболее слабая?

Мощность шкалы – степень ее возможностей для точного описания явлений. Всегда можно перейти от более мощной шкалы к более «слабой».

ВЫВОДЫ (ОСНОВНЫЕ МОМЕНТЫ ЛЕКЦИИ)

1. МО занимается поиском знаний в массивах данных.
2. Особенность МО: при увеличении объема данных точность решения задач должна увеличиваться.
3. В МО используется индуктивное обучение (обучение по прецендентам).
4. В МО решаются не только задачи анализа, но и предварительной обработки данных.

Машинное обучение
Лекция 2
Подготовка данных

Подготовка данных I

Подготовка данных

Цель – получение итогового набора данных, которые будут использоваться при моделировании, из исходных разнородных и разноформатных данных. Задачи подготовки данных могут выполняться **много раз без какого-либо наперед заданного порядка**. Они включают в себя отбор таблиц, записей и атрибутов, а также конвертацию и очистку данных для моделирования.

Шаги этапа:

1. Отбор данных (Select data).
2. Очистка данных (Clean data).
3. Получение производных данных (Construct data).
4. Объединение данных (Integrate data).
5. Приведение данных к необходимому формату (Format data).

Основная причина необходимости этапа подготовки данных:

данные обычно получаются из разных источников: из ERP- и CRM-систем, из хранилищ данных, и открытых баз данных, вручную.

Подходы к подготовке данных:

- автоматический (имеющимися средствами ПО);
- скриптовый (разработанный специалистом по анализу данных конкретно для его данных);
- ручной.

Обычно – комбинирование всех этих подходов.

Отбор данных

Цель:

отбор данных для анализа.
Включает: выбор признаков (столбцов) и выбор объектов (строк) в таблице.

Критерии:

- релевантность для целей анализа данных;
- качество;
- технические ограничения (объем данных, тип данных).

Правила включения/исключения (Rationale for inclusion/exclusion)

Необходимо перечислить данные, которые должны быть включены/исключены и по каким причинам.

Термин для исходных данных:

raw data (сырые данные).

Очистка данных I

Цель:

приведение качества данных в соответствие с выбранным аналитическим подходом.

Подходы:

- выбор достаточно качественного поднабора данных;
- анализ недопустимых значений;
- ввод значений по умолчанию (например, 0);
- орфографические ошибки и опечатки;
- удаление устаревших данных;

- оценка и восстановление пропусков в данных (мат. методы);
- удаление дубликатов (внимание на признаки, которые д.б. уникальными);
- анализ и удаление противоречивых записей (несколько записей для объекта);
- анализ и удаление аномалий.

Простейший подход поиска аномалий –

анализ «ящика с усами» (аномалии – все, что за пределами «усов»).
Межквартильный размах:

$$IQR = Q_3 - Q_1.$$

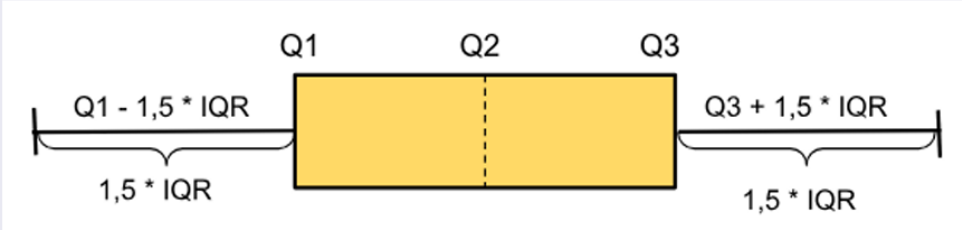


Рис.. Вид «ящика с усами»

Рекомендации:

1. удаление лишних исходных или производных данных (дата рождения и возраст);
2. если некоторые признаки постоянны или обладают **малой вариативностью**, имеет смысл их удалить;
3. оставление 1 признака из набора признаков с высоким уровнем корреляции (например, при $|corr(x, y) \geqslant 0,99|$.

*Анализ важности признаков (features importance) (после построения модели)

Слабые информационные признаки могут бессмысленно нагружать вычислительные ресурсы.
Самые важные признаки – возможно, в признаках есть целевая переменная. Важно понять причину их сильного влияния!

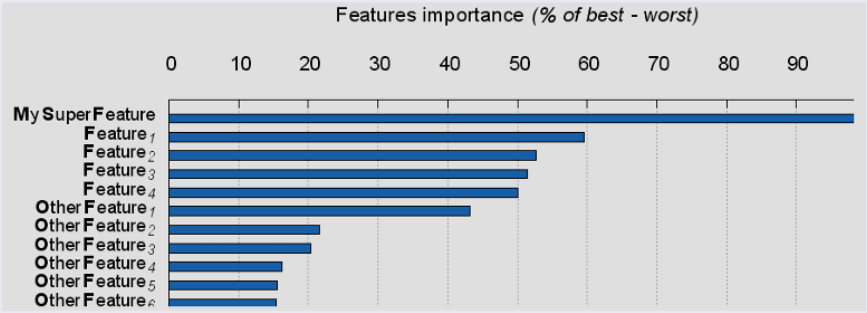


Рис.. Пример расчета важности признаков

В этом случае следует исключить признак из пространства.

Получение производных данных I

Цель:

операции по конструктивной подготовке данных (порождение новых признаков и новых записей, простое преобразование значений признаков).

Задачи:

- порождение признаков – создание одного и и более признаков из одной записи (строки) (длина*ширина –> площадь; ФИО -> <Фамилия, Имя, Отчество>);
- создание записей (строк) (новые записи для клиентов, которые не делали покупок последний год).

Обогащение данных (Data enrichment) –

дополнение потенциально полезной информацией из других источников.

Аугментация («раздутие») данных (Data augmentation) –

тиражирование похожих данных (для изображений – сдвиги, повороты, отражение, другие цвета).

Рекомендации:

1. кодирование категориальных данных: label encoding (нумерация целыми числами), one-hot encoding (создается бинарный вектор), binary encoding (двоичное представление label encoding);
2. стандартизация данных (приведение к нулевому среднему и стандартному отклонению, равному единице);
3. нормализация данных (если разные признаки имеют разные масштабы, например в диапазон $[0; 1]$ или $[-1; 1]$);
4. при больших разбросах в диапазоне значений признака – **логарифмирование** значений (характеристика порядка);
5. при периодичности – использование **тригонометрических** функций.

Объединение данных

Цель:

комбинирование информации из множества таблиц, отчетов, протоколов, анкет.

Слияние таблиц данных –

объединение двух и более таблиц, которые содержат различную информацию об одних и тех же объектах (могут быть объединены таблицы в новую с одной записью для каждого магазина с его характеристиками, комбинированными атрибутами по продажам и данных о покупателях из других таблиц).

Основной метод –

JOIN-запрос к БД.

Слияние данных включает агрегацию –

суммирование, усреднение информации из множества записей или таблиц (по каждому потребителю - суммарная по всем его покупкам - общее количество, общая стоимость, средняя стоимость).

Приведение данных к необходимому формату I

Цель:

синтаксические изменения, не меняющие значения данных. Могут определяться требованиями инструментов моделирования.

Переформатирование данных:

учет требований к порядку атрибутов (первое поле – индентификатор для каждой записи, последнее поле – результат, который должна предсказать модель). Для алгоритма моделирования может требоваться случайный порядок (атавизм?).

Синтаксические изменения:

- переименование данных;
- удаление ненужных знаков препинания в текстовых полях;
- урезание длинных слов до некоторого максимального заданного количества символов;
- преобразование номера телефона к международному формату (+79091234567);
- приведение изображений к единому формату (размер, черно-белый формат);
- устранение многозначности – преобразование к единому формату («город», «гор.», «г.»; «картофель», «картошка»);
- единый формат даты;
- приведение к единой единице измерения (г, кг, т);
- сохранение в необходимом формате (.xls, .csv).

Понятие и инструменты ETL/ELT I

ETL/ELT – способы доставки и интеграции данных из одного или нескольких источников в централизованное хранилище данных.

Цель ETL/ELT –

повышение степени доступности данных, удобства доступа, анализа данных.

Результат –

форма для РСУБД, обеспечение возможности запросов (SELECT) и OLAP.

Этапы:

- Extraction (извлечение);
- Transformation (преобразование);
- Loading (загрузка).

Разница ETL и ELT –

последовательность действий, которая приводит к разным затратам ресурсов.

ETL:

данные преобразуются до того, как попадут в ХД. Возникновение – 1970-е годы.

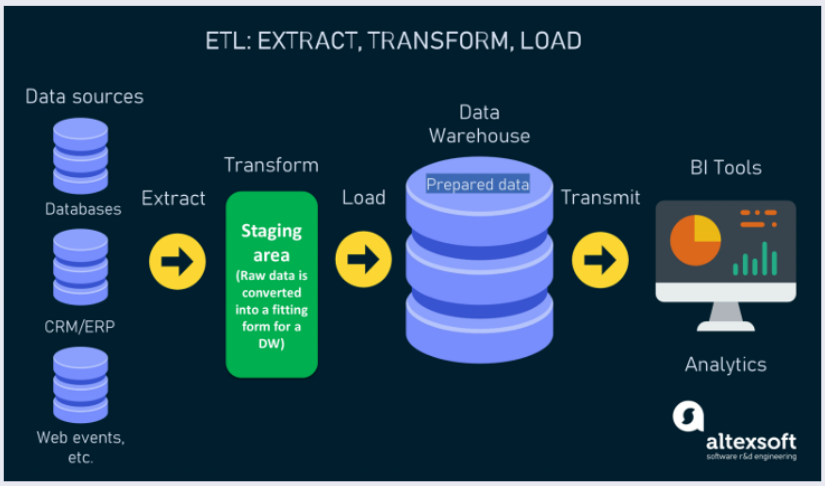


Рис.. Схема процесса ETL

Проблемы ETL:

- рост объемов данных –> больше ресурсов из-за промежуточного хранения данных (staging area);
- рост объема неструктурированных данных;
- нельзя преобразовать уже загруженные данные.

ELT:

преобразование производится после их загрузки в ХД (с начала 2000-х гг.).

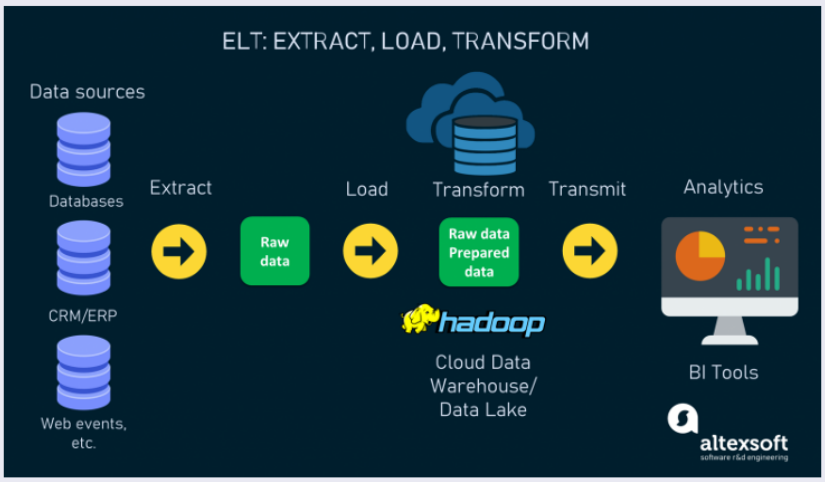


Рис.. Схема процесса ELT

Извлечение

Источники:

- ERP и CRM;
- баз данных SQL и NoSQL;
- приложения SaaS;
- веб-страницы;
- неструктурированные файлы;
- электронные письма;
- мобильные приложения.

Способы извлечения:

- полное: если системы не могут отличать новые/изменённые записи;
- частичное (с уведомлениями об обновлениях): если в системах есть уведомления об изменениях записей (самый удобный способ извлечения данных);
- инкрементное (частичное извлечение без уведомлений об обновлениях): извлечение тех записей, которые были изменены.

Преобразования:

- сортировка, фильтрацию данных;
- устранение дубликатов и очистка данных;
- транслирование и преобразование данных;
- удаление или шифрование для защиты уязвимой информации;
- соединение или разделение таблиц.

Загрузка

импорт ранее извлечённых (ETL: и уже подготовленных данных из промежуточной базы) в ХД.
Способы:

- вставка при помощи INSERT команд SQL;
- загрузка с помощью сценария (скрипта) пакетной загрузки большого объёма данных.

Сравнение ETL и ELT:

- различные методы подходят в разных случаях использования;
- ETL надёжнее и взрослее: инструменты и компетенции ETL на рынке представлены шире;
- ELT превосходит с точки зрения возможностей обработки больших массивов различных типов данных;
- ELT относительно менее затратен по сравнению с ETL на мощностях компании;
- ELT обеспечивает более быструю загрузку;
- при ELT преобразования занимают меньше времени;
- ETL имеет преимущество с точки зрения безопасности передаваемых данных.

Инструменты ETL и ELT

Talend Open Studio (TOS, создан на базе Eclipse) –

ETL-решение.

- Открытый исходным кодом.
- Графический пользовательский интерфейс.
- Позволяет создавать конвейеры данных и управлять ими.
- Библиотека компонентов позволяет подключаться к разным источникам данных: базам данных, файлам, веб-службам.
- Функции очистки и преобразования данных.

DBT (Data Building Tool, написан на Python) –

T-инструмент из ELT. Инструмент командной строки для преобразования данных в своих хранилищах.

- Задача – компиляция кода DBT в SQL-запрос и выполнение его.
- Требуется понимание только SQL и Python.
- CLI (консольная) версия: функции для преобразования данных – запуск тестов, компиляция, создание документации;
- UI версия: не дает возможности изменить потоки данных, используется в основном для документирования.

Apache Airflow (написан на Python) –

ETL/ELT-решение.

- Сбор данных из различных источников.
- Планирование сценариев создания и обновления своих наборов данных на основе загружаемых из любых наборов данных Airflow.
- Хранилище объектов – упрощает взаимодействие с системами хранения объектов, такими как Amazon S3, Google Cloud Storage и Azure Blob Storage.
- Множество расширений Airflow для упрощения интеграции, расширения функционала.

Apache NiFi –

ETL/ELT-решение. Конструктор для создания пайплайнов загрузки данных.

- Позволяет подготовить типовые пайплайны загрузки данных и многократно использовать их.
- Пайплайны можно группировать.
- Дружелюбный интерфейс (в отличие от кода в Apache Airflow).
- Защита от случайных действий пользователя.
- Возможна работа пользователей с минимальными навыками.
- Отслеживание изменений и их последствий.

Заключение

1. Подготовка данных – важный этап анализа данных и последующего применения методов машинного обучения.
2. Подготовка включает решение задач, которые могут решаться в любой последовательности: отбор данных, очистка данных, получение производных данных (Construct data), объединение данных (Integrate data), приведение данных к необходимому формату.
3. Близкими методиками являются процессы ETL/ELT, отличие которых не только в порядке следования этапов.

Машинное обучение
Лекция 3
Методы классификации данных

2.1. Постановка задачи классификации I

Входные данные:

1. Множество векторов признаков (features):

$$\{x^{(i)}\}, x^{(i)} = \begin{bmatrix} x_1 & x_2 & \dots & x_n \end{bmatrix}^T \in X \subset \mathbb{R}^n, i = 1, \dots, m,$$

   X – пространство объектов (образов) классификации (распознавания).
2. Множество непересекающихся классов (меток):

$$C = \{C_j\}, j = 1, \dots, k.$$

   Обычно $C_j = j, j = 1, \dots, k$.
3. Множество классов для известных объектов:

$$\{C^{(i)}\}, i = 1, \dots, m.$$

   Обычно $C_j = j, j = 1, \dots, k$.

Множество пар $\{x^{(i)}, c^{(i)}\}, i = 1, \dots, m$ – обучающее множество.

Часто предполагают:

$X = \mathbb{R}^n$ – евклидово пространство.

Отличие от задачи **регрессии**: множество выходных значений $\{C_j\}$ – дискретно. Классы соответствуют номинальным переменным.

Цель:

определение $f : X \to C$.

Классификация – задача обучения с учителем (по прецедентам).
Альтернатива: поиск поверхностей, разбивающих пространство $\mathbb{R}^n$ на k непересекающихся подпространств.
Замечание: в $\mathbb{R}^2$ – поиск кривых.

Простейший случай:

Бинарная классификация ($k = 2$). Обычно: $C_0 = 0$ (негативный класс, «-»), $C_1 = 1$ (позитивный класс, «+»).

Варианты постановки задачи:

1. Определение ВФВА $f : X \to C^k$, $C_j \in \{0, 1\}$
2. Определение ВФВА $f : X \to C^k$, $C_j \in [0, 1]$, вероятность отнесения к классу j.

Линейная разделимость:

$$f = \sum_{i=0}^{n} a_i x_i \geqslant (\leqslant) T$$

2.2. Точность классификации I

Матрица классификации (матрица потерь, confusion matrix) на тестовом множестве:

$$\begin{bmatrix} \text{Известные - Предсказанные} & C_1 & C_2 & \dots & C_k \\ C_1 & t_{11} & e_{12} & \dots & e_{1k} \\ C_2 & e_{21} & t_{22} & \dots & e_{2k} \\ \dots & \dots & \dots & \dots & \dots \\ C_k & e_{k1} & e_{k2} & \dots & t_{kk} \end{bmatrix}$$

Диагональные элементы – правильные ответы; внедиагональные – ошибки
Пример:

$$\begin{bmatrix} \text{Известные - Предсказанные} & C_1 & C_2 & \dots & C_k \\ C_1 & \mathbf{95} & 4 & \dots & 1 \\ C_2 & 3 & \mathbf{93} & \dots & 4 \\ \dots & \dots & \dots & \dots & \dots \\ C_k & 0 & 2 & \dots & \mathbf{98} \end{bmatrix}$$

Точность (accuracy):

$$Acc = \frac{\sum_{i=1}^{k} t_{ii}}{m},$$

где $m = \sum_{i=1}^{k} t_{ii} + \sum_{i=1}^{k} \sum_{p=1, p \neq i}^{k} e_{ip}$ – общее количество элементов обучающего множества.

Уровень ошибки (error rate):

$$Err = \frac{\sum\limits_{i=1}^{k} \sum\limits_{p=1, p \neq i}^{k} e_{ip}}{m}$$

Точность (precision) классификации j-го класса:

$$Pr_j = \frac{t_{jj}}{t_{jj} + \sum\limits_{i=1, i \neq j}^{k} e_{ij}}$$

Чувствительность (recall, sensitivity) j-го класса:

$$Sens_j = \frac{t_{jj}}{t_{jj} + \sum\limits_{i=1, i \neq j}^{k} e_{ji}}$$

Специфичность (specificity) j-го класса (мера того, что объект не класс j):

$$Spec_j = \frac{tn_{jj}}{tn_{jj} + \sum\limits_{i=1, i \neq j}^{k} e_{ji}},$$

где $tn_{jj} = \sum\limits_{i=1, i \neq j}^{k} t_{ii} + \sum\limits_{i=1, i \neq j}^{k} \sum\limits_{p=1, p \neq j}^{k} e_{ip}$
Все эти показатели: $0 \leqslant \ldots \leqslant 1$.
Если $k = 2$: $Sens_1 = Spec_2$, $Sens_2 = Spec_1$.

Рассмотрим **бинарную классификацию**. Два класса: C_1 – положительный (positive), C_2 – отрицательный (negative). Обозначим:

- P – количество строк с положительным классом в обучающем множестве;
- N – количество строк с отрицательным классом в обучающем множестве.

Обозначения:

- TP, True Positive – верноположительный результат.
- FP, False Positive – ложноположительный результат.
- TN, True Negative – верноотрицательный результат.
- FN, False Negative – ложноотрицательный результат.

Соотношения:

- $P = TP + FN$;
- $N = TN + FP$.

Матрица классификации:

$$\begin{bmatrix} \text{Известные - Предсказанные} & \textit{Positive} & \textit{Negative} \\ \textit{Positive} & \textit{TP} & \textit{FN}(\text{ош. 2 рода}) \\ \textit{Negative} & \textit{FP}(\text{ош. 1 рода}) & \textit{TN} \end{bmatrix}.$$

Примечание: часто (в частности, в источниках ниже) используется транспонированная матрица.

Чувствительность j-го класса (см. определение):

$$Sens_j = \frac{t_{jj}}{t_{jj} + \sum\limits_{i=1, i\neq j}^{k} e_{ji}}.$$

В бинарной классификации – **точность** и **полнота** относятся к положительному классу.

Точность = степень правильности положительных (TPR, True Positive Rate):

$$Precision = TPR = \frac{TP}{TP + FP}.$$

Полнота:

$$Recall = \frac{TP}{P} = \frac{TP}{TP + FN}.$$

Определение специфичности (см. ранее)

$$Spec_j = \frac{tn_{jj}}{tn_{jj} + \sum\limits_{i=1, i\neq j}^{m} e_{ji}},$$

где $tn_{jj} = \sum\limits_{i=1, i\neq j}^{m} t_{ii} + \sum\limits_{i=1, i\neq j}^{m} \sum\limits_{p=1, p\neq j}^{m} e_{ip}$.

Степень правильности отрицательных (TNR, True Negative Rate):

$$TNR = \frac{TN}{TN + FN}.$$

В бинарной классификации – **специфичность** относится к отрицательному классу:

$$Spec = \frac{TN}{N} = \frac{TN}{TN + FP}.$$

Степень ложности положительных (FPR, False Positive Rate):

$$FPR = \frac{FP}{FP + TN} = 1 - TNR.$$

Медицина.

- P – Больной человек: положительный (с точки зрения наличия болезни) класс.

- N – Здоровый человек: отрицательный (с точки зрения наличия болезни) класс.
- TP – Больные люди, правильно определенные как больные.
- FP – Здоровые люди, неверно определенные как больные.
- TN – Здоровые люди, правильно определенные как здоровые.
- FN – Больные люди, неверно определенные как здоровые.

Ситуация: испытание нового метода диагностики заболевания. Методика: 2 группы людей заведомо больных (с подтвержденным диагнозом, $P = 100$) и заведомо здоровых (с отвергнутым диагнозом, $N = 100$).
Если с помощью нового метода выявлено 90 больных (TP, истинно положительный результат диагностики), значит, чувствительность проверяемого метода равна 90%. Чувствительность метода диагностики – вероятность **выявить заболевание у больных людей**.
Пусть при обследовании здоровых проверяемым методом у 95 человек болезнь отвергнута (TN, истинно отрицательный результат диагностики), значит, специфичность метода равна 95%. Специфичность – вероятность **определения отсутствия заболевания у здоровых людей**.
Матрица классификации:

$$\begin{bmatrix} \text{Известные - Предсказанные} & \textit{Positive} & \textit{Negative} \\ \textit{Positive} & TP = 90 & FN = 10 \\ \textit{Negative} & FP = 5 & TN = 95 \end{bmatrix}$$

Отличие полноты (чувствительности) (90%) и точности (положительного предсказывающего значения) теста (точность определения положительного класса) ($TPR = \frac{90}{90+5} \approx 94,7\%$).
Чувствительность теста – вероятность выявления заболевания у действительно больного.
Положительное предсказывающее значение теста – вероятность наличия заболевания при положительном результате теста.

F-мера (F-score)

Оптимальный баланс между метриками точности (P) и полноты (R)– F-мера – гармоническое среднее между точностью и полнотой:

$$F_\beta = \frac{(1+\beta^2)PR}{\beta^2 P + R},$$

где $\beta \geqslant 0$ – параметр меры:

- $0 \leqslant \beta < 1$ – приоритет точности;
- $\beta > 1$ – приоритет полноте;
- $\beta = 1$ – сбалансированная F-мера (F_1):

$$F_1 = \frac{2PR}{P+R}.$$

ROC-кривая (Receiver Operating Characteristic curve) –

график с координатными осями Ox: FPR; Oy: TPR.

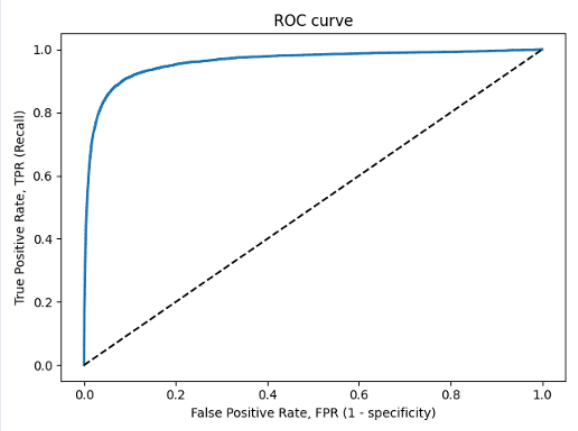


Рис.. ROC-кривая

Прямая линия по диагонали – ROC-кривая случайного классификатора. Хороший классификатор – стремится к левому верхнему углу.
Площадь под кривой – **ROC-AUC** (Area Under the Curve).

- $ROC - AUC = 1$ – идеально;
- $ROC - AUC = 0,5$ – случайный классификатор;
- $ROC - AUC = 0$ – предсказание классов наоборот.

Если $P << N$: то ROC-AUC – неадекватная оценка качества работы алгоритма, поскольку измеряет долю неверно принятых объектов относительно общего числа отрицательных.

PR-кривая (Precision-Recall curve) –

график с координатными осями Ox: Recall; Oy: Precision.

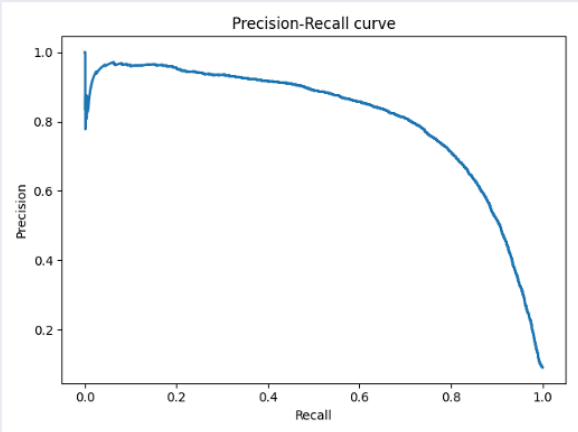


Рис.. PR-кривая

Для **несбалансированных** классов.
Площадь под кривой – **PR-AUC**.

- $PR - AUC = 1$ – идеально;
- $PR - AUC \to 0$ – слабая способность прогнозировать класс P.

Обобщение-1:

$$L(x^{(i)}) = \{f(x^{(i)}) \neq C^{(i)}\}.$$

Средняя ошибка:

$$R = \frac{1}{m}\sum_{i=1}^{m} L(x^{(i)}).$$

Задача: $R \to \min$

Обобщение-2:

Могут задаваться потери от ошибок отнесения объекта i к классу j : λ_{ij}. Обычно: $\lambda_{jj} = 0$, $\lambda_{ij} > 0$ при $i \neq j$.

Роды ошибок:

- 1-го рода: объект j считается отнесенным к другому классу (пропуск цели);
- 2-го рода: объект i считается отнесенным к классу j (ложная тревога).

Вопрос:

как соотносятся ошибки потерь для классификаций «самолет» – «птица» и «спам» – «обычные сообщения» для пропуска цели и ложной тревоги?

Оценка адекватности модели классификации:

- на основе оценки классификации на тестовом множестве;
- если результаты на обучающем и тестовом множестве прибл. равны;
- кросс-проверка (Cross-Validation) (на маленьком множестве) – метод скользящего экзамена.

Многоклассовая классификация (на основе бинарной):

обучается несколько моделей, каждая из которых отличает целевой класс от остальных.

2.3. Наивный байесовский классификатор I

Naive Bayes Classifier

Вероятность того, что объект x принадлежит классу C_j:

$$P(C_j : x), j = 1, \ldots, k$$

Это апостериорная вероятность.

Байесовское решающее правило: объект относится к классу j^*, если

$$j^* = \arg\max_{j=1}^{m} P(C_j : x)$$

Это принцип максимума апостериорной вероятности; maximum a posteriori hypothesis, MAP.
В общем случае:

$$j^* = \arg\max_{j=1}^{m} \lambda_j P(C_j : x),$$

где λ_j – значимость классов.
Для $k = 2$: если $P(C_1 : x) > P(C_2 : x)$, то класс C_1 иначе класса C_2.
Используется формула Байеса:

$$P(C_j : x) = \frac{P(C_j, x)}{\sum_{i=1}^{m} P(C_i)P(x : C_i)} = \frac{P(C_j)P(x : C_j)}{\sum_{i=1}^{m} P(C_i)P(x : C_i)},$$

где $P(C_j)$ – априорная вероятность события, $P(x : C_j)$ – условная вероятность события x при условии, что произошло C_j.
Оценка априорных вероятностей появления классов по прецедентам:

$$P(C_j) \approx \frac{m_j}{m},$$

где m_j – количество примеров из обучающего множества с классом C_j.
Оценка функций правдоподобия:

$$P(x : C_j) = P(x_1 : C_j) \cdot P(x_2, \ldots, x_n : C_j, x_1) = P(x_1 : C_j) \cdot P(x_2 : C_j, x_1) \times$$

$$\times P(x_3, \ldots, x_n : C_j, x_1, x_2) = \ldots = P(x_1 : C_j) \cdot P(x_2 : C_j, x_1) \cdot P(x_3 : C_j, x_1, x_2) \cdot \ldots \times$$

$$\times P(x_n : C_j, x_1, x_2, \ldots, x_{n-1}).$$

Если признаки независимы (наивный байесовский классификатор), то:

$$P(x : C_j) = \prod_{i=1}^{n} p(x_i : C_j).$$

Оценка – по гистограммам частот (куда попадут значения x_i вектора x).
Для номинальных шкал:

$$p(x_i : C_j) \approx \frac{|x_i(C_j)|}{|C_j|},$$

где $|C_j| = m_j$ – число объектов класса j; $|x_i(C_j)|$ – количество появлений значения x_i в классе C_j.
Для непрерывных шкал: переход к интервалам изменений значений переменных.
Прим.: знаменатель везде одинаков! Достаточно сравнивать числители.
Если априорные вероятности $P(C_j)$ одинаковы, прицип максимума значительно упрощается:

$$j^* = \arg\max_{j=1}^{m} P(x : C_j).$$

Предположения:

1. существование распределения вероятностей для каждого класса и функций правдоподобия;
2. условие независимости для функций правдоподобия.

Наивный байесовский классификатор часто используется как шаблон на модельных данных для сравнения других методов классификации.
Алгоритм применения:

1. для каждого класса вычисляется апостериорная вероятность (возможно, с учетом замечаний);
2. выбирается тот класс, для которого значение максимально.

Переход к логарифмам

Проблема: много признаков – много произведений маленьких чисел – потеря точности.
Решение: переход к логарифмам ($\log(x)$ – монотонно возрастающая; $\log ab = \log a + \log b$). Основание значения не имеет.
Тогда:

$$\log P(C_j)P(x : C_j) = \log P(C_j) + \log P(x : C_j) = \log P(C_j) + \sum_{i=1}^{n} \log p(x_i : C_j).$$

Проблемы

Если номинальная шкала, и в обучающем множестве значение x_i не встречалось. Тогда все апостериорные вероятности $= 0$.
Решение: аддитивное сглаживание (сглаживание Лапласа):

$$p(x_i : C_j) \approx \frac{|x_i(C_j)| + 1}{|C_j| + w_i(C_j)},$$

где $w_i(C_j)$ – количество разных значений переменной i в классе C_j.

ВЫВОДЫ (ОСНОВНЫЕ МОМЕНТЫ ЛЕКЦИИ)

1. Задача классификации – часто встречаемая задача, связанная с необходимостью отнесения объекта к определенному известому классу.
2. Для оценки точности исходной информацией является матрица потерь.
3. Возможно задание точности классификации в зависимости от решаемой задачи.
4. Базовый алгоритм классификации – наивный байесовский классификатор.
5. Важнейшая задача при использовании Н.Б.К. – восстановление плотности распределения.
6. Существенное предпложение Н.Б.К. – независимость переменных.

Машинное обучение
Лекция 4
Деревья решений. Метод опорных векторов

Логистическая регрессия I

Используется сигмоидная логистическая функция

$$\sigma(y) = \frac{1}{1 + e^{-y}} \in (0; 1).$$

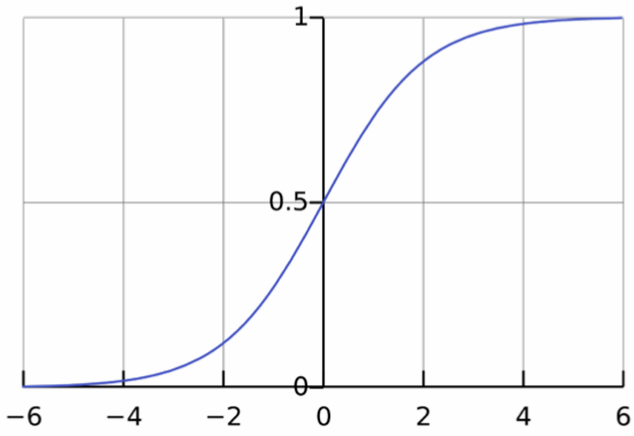


Рис.. Сигмоидная логистическая функция

Здесь y часто называется **logit**. Если известно $p = \sigma(y)$, то

$$\text{logit}(p) = \sigma^{-1}(p) = \ln \frac{p}{1 - p}.$$

Значение y – простейшая линейная модель

$$y = \sum_{i=0}^{n} w_i x_i,$$

поэтому:

$$\sigma(y) = \frac{1}{1 + e^{-\sum_{i=0}^{n} w_i x_i}} = \frac{1}{1 + e^{-w_0 - w_1 x_1 - \ldots - w_n x_n}}.$$

Решающая граница (decision boundary) – $0 < \theta < 1$ – порог:

- $\sigma(y) > \theta$ – $x \in C_1$;
- $\sigma(y) \leqslant \theta$ – $x \in C_0$.

Обычно $\theta = 0,5$.

$$p_1 = \sigma(y) = \frac{1}{1 + e^{-y}};$$

$$p_0 = 1 - p_1 = 1 - \frac{1}{1+e^{-y}} = \frac{e^{-y}}{1+e^{-y}}.$$

Т.к. для сигмоиды

$$1 - \sigma(y) = \sigma(-y),$$

то

$$p_0 = \sigma(-y).$$

В отличие от наивного байесовского классификатора классификация на оснвое логистической регрессии более робастна к коррелированности признаков.

Ошибка «Log Loss» (Logarithmic Loss) –

$$\text{Log Loss} = -\frac{1}{k}\sum_{i=1}^{k}(y_i \log(p_i) + (1-y_i)\log(1-p_i)) \to \min,$$

где p_i – вероятность того, что объект относится к классу P.

- Log Loss $= 0$ – идеально;
- Log Loss $\to \infty$– неверные предсказания.

Используется не для отдельной модели, а для сравнения различных моделей.

Деревья решений I

Деревья решений (decision trees) –

наиболее распространенный метод классификации (Giovanni Seni and John Elder, Ensemble Methods in Data Mining: Improving Accuracy Through Combining Predictions, Synthesis Lectures on Data Mining and Knowledge Discovery (Morgan and Claypool, 2010), 28).

Первая работа –

Ховиленд, Хант (Hoveland, Hunt) «Эксперименты в индукции» («Experiments in Induction»), 1966 г.

Базовый вариант –

представление правил с помощью иерархической, последовательной структуры **бинарного дерева**. Основа – переход к ветвям в зависимости от ответов «Да» или «Нет».

Алгоритм применения: движение от корня дерева (корневой вершины) до его листа. В каждой вершине (узле) осуществляется развилка. Каждый **лист** (конечный узел, terminating block) – класс C_i. В общем случае из узла – несколько ($\geqslant 2$) ветвей.

Внутренние узлы (возраст, наличие недвижимости, доход,образование) – **атрибуты расщепления** (splitting attributes).
Каждая ветвь дерева, идущая от внутреннего узла, отмечена **предикатом расщепления**.
Информация об атрибутах расщепления и предикатах расщепления в узле – **критерий расщепления (бок решений)** (splitting criterion (decision block)).

Деревья решений – удобный способ представления **продукционной модели знаний**.

Преимущества:

интуитивно понятны, интерпретируемы, эффективны.

Проблема выбора оптимального дерева

На обучающем множестве: чем больше описано частных случаев, тем меньшее количество объектов попадает в каждый частный случай. Много узлов и ветвей – «ветвистые» («кустистые») деревья. Следствие: низкая способность к обобщению. Недостаток – **переобучение (overfitting)**.
Оптимальное дерево: достаточно сложное, чтобы учитывать информацию из обучающего множества, но одновременно должно быть достаточно простым.
Две стратегии:

1. наращивание дерева до определенного размера в соответствии с параметрами, заданными пользователем;
2. сокращение дерева путем отсечения ветвей и использование правил остановки обучения.

Основная задача:

разработка алгоритмов построения оптимальных деревьев решений на основе множества известных данных.

Алгоритмы построения: ID3, CART, C4.5, CHAID, See5, Sipina, CN2, NewId, ITrule...
Основной принцип: «разделяй и властвуй» (divide and conquer).
Идея – рекурсивное разбиение обучающего множества на подмножества, чтобы в каждом подмножестве доминировал (властвовал) один из классов.

Задача:

выбирать параметры разбиения на каждом шаге, чтобы обеспечить преобладание (лучше: **полное присутствие**) объектов одного класса.

1. Выбирается переменная, которая помещается в корень дерева.
2. Из вершины строятся ветви, соответствующие всем возможным значениям выбранной независимой переменной.
3. Множество объектов из обучающей выборки разбивается на несколько подмножеств в соответствии со значением выбранной переменной.

Возможные ситуации

относительно обучающего множества и множества классов C:

1. множество содержит один или более объектов, относящихся к **одному** классу C_i. Тогда дерево решений – лист, определяющий класс C_i;
2. множество не содержит ни одного объекта (пустое множество). Это лист с классом, который выбирается из другого множества, отличного от текущего, например из множества, ассоциированного с родителем;
3. множество содержит объекты, относящиеся к разным классам. Выбирается одна из переменных x_i, имеющая два и более отличных друг от друга значений $x_{i_1}, x_{i_2}, \ldots, x_{i_k}$. Множество разбивается на k подмножеств, каждое из которых содержит все объекты, у которых значение выбранной переменной равно x_{i_k}. Процесс продолжается рекурсивно для каждого подмножества до тех пор, пока значение переменной во вновь образованном подмножестве не будет одинаковым (когда объекты принадлежат одному классу). В этом случае процесс для данной ветви дерева прекращается.

C4.5 – алгоритм построения дерева решений для задач классификации, количество потомков у узла не ограничено. Предложен Р. Куинленом (R. Quinlan).

Особенности:

- Количество переменных должно быть фиксированным для всех примеров.
- Каждая переменная иметь номинальное или непрерывное* значение.
- Количество классов должно быть значительно меньше количества примеров.

Исходные данные:

- Обучающее множество X объема $|X|$, n переменных для описания объектов.
- Классы $C_1, C_2, \ldots, C_k$.

Номинальные значения

Начальная ситуация:

Пустое дерево, включающее только корень. Множество X **ассоциировано** с ним.

Задача:

выбор переменной, по которой должно пойти ветвление. Всего n вариантов (т.к. n переменных). Необходим **критерий**.

Важная особенность алгоритма:

любая из переменных можно использовать **неограниченное количество раз** при построении дерева.

Пусть выбрана переменная $\tilde{x}$, принимающая p различных значений $\tilde{x}_1, \tilde{x}_2, \ldots, \tilde{x}_p$. Будет p разбиений на множества $X_1, X_2, \ldots, X_p$.

Обозначим количество примеров во множестве X, относящихся к классу C_j:

$$freq(C_j, X).$$

Вероятность того, что случайно выбранный пример из множества X будет принадлежать к классу C_j:

$$p_j = \frac{freq(C_j, X)}{|X|}.$$

Энтропия множества X (из **теории информации**):

$$Info(X) = -\sum_{j=1}^{k} p_j log_2 p_j.$$

Это оценка среднего количества информации, необходимого для определения класса примера из множества X.
Энтропия после разбиения множества X:

$$Info_{\tilde{x}}(X) = \sum_{i=1}^{p} \frac{|X_i|}{|X|} \cdot Info(X_i).$$

Критерий выбора переменной (в алгоритме ID3):

$$Gain(\tilde{x}) = Info(X) - Info_{\tilde{x}}(X).$$

Этот критерий рассчитывается для всех переменных. Выбирается переменная, для которой критерий **максимален** (несет большее количество информации).

Смысл информационного критерия:

при разбиении по переменной один из классов будет иметь наибольшую вероятность появления (когда значение $Info_{\tilde{x}}(X)$ будет минимально).

Осуществляется рекурсивный процесс построения дерева, до тех пор, пока в узле не окажутся примеры из одного класса.

Замечание:

если в процессе работы получен узел, ассоциированный с пустым множеством (т.е. ни один пример не попал в данный узел), то он помечается как лист. В качестве решения листа выбирается наиболее часто встречающийся класс у непосредственного предка данного листа.

Вещественные* значения

Пусть переменная имеет конечное* (в связи с конечностью выборки; потенциально - бесконечное количество значений) число значений. Строится **бинарное** дерево.
Обозначим значения: $\{v_1, v_2, \ldots, v_p\}$. Отсортируем все значения. Выберем порог, с которым должны сравниваться все значения переменной. Порог, лежащий между v_i и v_{i+1}, делит все примеры на два множества: меньше порога ($\{v_1, v_2, \ldots, v_i\}$), и больше порога ($\{v_{i+1}, v_{i+2}, \ldots, v_p\}$).

Порог –

$$T_i = \frac{v_i + v_{i+1}}{2}.$$

В результате для переменной с p значениями получен $(p-1)$ порог $(T_1, T_2, \ldots, T_{p-1})$.
Для каждого порога переменной высчитывается критерий; выбирается максимальный. Далее он сравнивается с критериями для други переменных.

Замечание:

все тесты являются бинарными, т.е. делят узел дерева на две ветви.

Проблема:

Недостаток критерия – «предпочитает» переменные с большим числом значений. Пусть одна из переменных идентифицирует личность пациента (ФИО, паспорт, табельный номер). Т.к. каждое значение уникально, то при разбиении множества примеров по этому атрибуту получаются подмножества, содержащие **только по одному примеру**. Так как все эти множества «однопримерные», то и пример относится к одному (единственному) классу, поэтому

$$Info_{\tilde{x}}(X) = 0.$$

Будет выборана именно эта переменная. Такое дерево – бесполезно для классификации.

Проблема решается введением некоторой нормализации. По аналогии с определением $Info(X)$:

$$splitInfo(\tilde{x}) = -\sum_{i=1}^{p} \frac{|X_i|}{|X|} log_2 \frac{|X_i|}{|X|}.$$

Это оценка потенциальной информации, получаемую при разбиении множества X на p подмножеств.

Новый критерий, используемый в C4.5:

$$gainRation(tildex) = \frac{Gain(\tilde{x})}{splitInfo(\tilde{x})}.$$

Переменная-идентификатор не будет высоко оценена новым критерием.

Эвристическое улучшение,

чтобы не создавать узлы и листья с незначительным количеством примеров: при разбиении множества X, по крайней мере два подмножества должны иметь не меньше заданного минимального количества примеров $q > 1$ (обычно $q = 2$). В случае невыполнения правила дальнейшее разбиение множества прекращается; соответствующий узел отмечается как лист. При таком ограничении может возникнуть ситуация, когда примеры, ассоциированные с узлом, относятся к разным классам. Тогда выбирается класс, который наиболее часто встречается в узле, если же примеров равное количество из всех классов, то решение дает наиболее часто встречающийся класс у непосредственного предка данного листа.

Случайный лес

Случайный лес деревьев решений – **ансамблевый** метод решения задач классификации. Обычно – **бэггинг**.
k – количество строк, n – количество признаков. Пусть $\hat{k} \leqslant k$; $\hat{n} < n$.
Количество деревьев (размер леса): t_n.
Алгоритм (цикл t_n раз):

1. генерация на основе обучающего множества подвыборку размером с возвращением $\hat{k} \times \hat{n}$ (некоторые примеры могут попасть несколько раз);
2. строится дерево решений – обычно небольшого размера.

Классификация объектов: **голосование** – каждое дерево относит объект к одному из классов; побеждает класс, за который проголосовало наибольшее число деревьев.
Оптимальное t_n может подбираться из минимизация ошибки на тестовой выборке.
Если дерево решений из 1 вершины – **лес пней**.

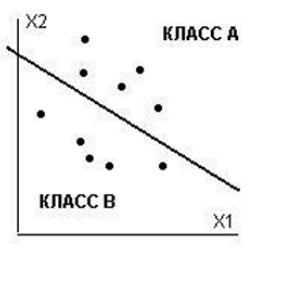
Метод опорных векторов (SVM) I

Метод опорных векторов (Support Vector Machine, SVM) –

алгоритмы решения задач **линейной** классификации на основе обучения по прецедентам с учителем.

Линейная классификация –

разделение множества объектов $\{x^{(i)}\}, x^{(i)} \in \mathbb{R}^n, i = 1, \ldots, m$, с помощью $(n-1)$-мерной гиперплоскости. Двумерный случай $(n = 2)$:



Примечание: аналогичную задачу решает **перцептрон**.
Замечание: если задача линейной классификации решается – **несколько решений (гиперплоскостей)**.

Основная идея:

гиперплоскость должна разделять классы **как можно лучше** (чтобы два класса лежали как можно дальше). Следствие: небольшие изменения в гиперплоскости не сильно изменят результат классификации.

Два класса ($C_1 = 1$, $C_2 = -1$):

Линейная классификация для объекта $x^{(i)}$:

$$f(x^{(i)}) = sign(\langle x, \hat{w} \rangle) = sign\left(\sum_{j=1}^{n} w_j x_j^{(i)} + w_0\right),$$

где w_j – вес j-го признака, w – весовой вектор из w_j, $-w_0$ – пороговое значение, $\hat{w}$ – расширенный вектор из w и w_0, $\langle x, \hat{w} \rangle$ – скалярное произведение,

$$sign(a) = \begin{cases} 1, & a \geqslant 0, \\ -1, & a < 0. \end{cases}$$

В идеальном случае:

1. $\forall x^{(i)} \in C_1$: $f(x^{(i)}) \geqslant 0 (\geqslant 1)$;
2. $\forall x^{(i)} \in C_2$: $f(x^{(i)}) < 0 (< -1)$.

Иначе: для всех $x^{(i)}, i = 1, \ldots, m$, должно выполняться условие

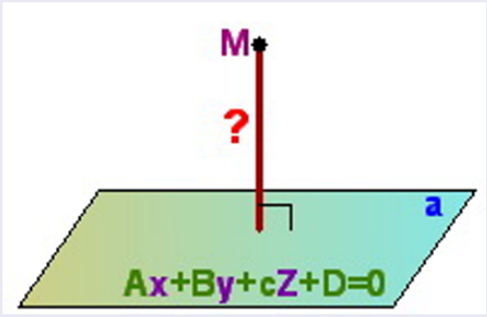
$$C^{(i)} \cdot f(x^{(i)}) \geqslant 0 (\geqslant 1). \tag{1}$$

Основной подход

Гиперплоскость – **опорная** для множества объектов X, все объекты из X лежат по одну сторону от нее.
Цель – максимизация **зазора** (margin) между двумя параллельными опорными плоскостями, далее провести параллельную им гиперплоскость на равных расстояниях от них.

Расстояние

от точки x до плоскости $f(x) = \langle x, \hat{w} \rangle = 0$: $\dfrac{|f(x)|}{\|w\|}$.



Постановка задачи максимизации зазора:

$$\arg\max_{w, w_0} \min_i \left[\frac{C^{(i)} \cdot f(x^{(i)})}{\|w\|}\right] =$$

$$= \arg\max_{w, w_0} \left\{\frac{1}{\|w\|} \cdot \min_i \left[C^{(i)} \langle x^{(i)}, \hat{w} \rangle\right]\right\}.$$

Здесь: $\min_i(\cdot)$ – расстояние от ближайшей точки до плоскости (зазор); $\max_w(\cdot)$ – максимизация зазора.

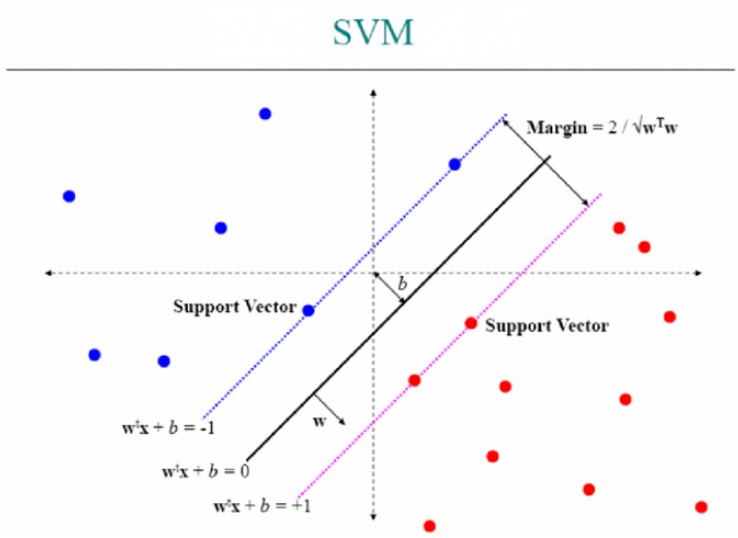
Все разделяющие плоскости:

$$\langle x, w \rangle + w_0', \quad w_0' \in (w_0 - 1, w_0 + 1).$$

Расстояние между граничными плоскостями:

$$\frac{2}{\|w\|}.$$

На граничных плоскостях могут находиться векторы из обучающего множества – **опорные векторы**.



Для повышения надежности $\|w\|$ должна быть как можно меньше.
В (1) для всех $x^{(i)}$:

$$C^{(i)} \cdot f(x^{(i)}) \geqslant 1. \tag{2}$$

Задача квадратичного программирования:

найти

$$\min_w \frac{1}{2}\|w\|^2$$

при условии (2) (выпуклый многогранник).

Двойственные задачи

Прямая задача:

$$\min f(x)$$

при условиях $h(x) = 0$, $g(x) \leqslant 0$, $x \in X$.
Двойственная задача:

$$\min \phi(\lambda, \mu), \quad \phi(\lambda, \mu) = \inf_{x \in X} \{f(x) + \lambda^T h(x) + \mu^T g(x)\}$$

при условиях $\mu \geqslant 0$.

Двойственная задача для SVM:

$$L(w, w_0, \alpha) = \frac{1}{2}\|w\|^2 - \sum_i \alpha_i \left[C^{(i)}(w^T x^{(i)} + w_0) - 1 \right], \quad (3)$$

где $\alpha_i \geqslant 0$ – множители Лагранжа.
Из НУО ($\frac{\partial L}{\partial w} = 0$ и $\frac{\partial L}{\partial w_0} = 0$):

$$w = \sum_i \alpha_i C^{(i)} x^{(i)};$$

$$\sum_i \alpha_i C^{(i)} = 0.$$

Подставляем в (3):

$$L(\alpha) = \sum_i \alpha_i - \frac{1}{2}\| \sum_i \alpha_i C^{(i)} x^{(i)} \|^2 \quad (4)$$

при условиях $\alpha_i \geqslant 0$, $\sum_i \alpha_i C^{(i)} = 0$.
Далее для классификации (В. Вапник, 1963 г.):

$$f(x) = \sum_i \alpha_i C^{(i)} (x^{(i)T} x^{(i)}) + w_0. \quad (5)$$

Замечание: суммирования в (4) и (5) не по всем векторам, а только по **опорным** (их значительно меньше m). Для опорных векторов:

$$C^{(i)} f(x^{(i)}) = 1.$$

Линейно-неразделимая задача:

введя неотрицательную ошибку (slack), найти

$$\min_w \left\{ \|w\|^2 + C \sum_{i=1}^{n} z_i \right\}$$

при условии

$$C^{(i)} \cdot f(x^{(i)}) + z_i \leqslant 1.$$

Двойственная задача:

$$\min_\alpha \left\{ \frac{1}{2}\| \sum_i \alpha_i C^{(i)} x^{(i)} \|^2 - \sum_i \alpha_i \right\}$$

при условиях $0 \leqslant \alpha_i \leqslant T$, $\sum_i \alpha_i C^{(i)} = 0$. Отличие от классического случая – верхняя граница T на влияние каждой точки.

ВЫВОДЫ (ОСНОВНЫЕ МОМЕНТЫ ЛЕКЦИИ)

1. Наиболее распространенный метод классификации – применение **деревьев решений**.
2. Основное преимущество деревьев решений – интуитивная понятность.
3. Принцип построения – **разделяй и властвуй**.
4. Подход: построение деревьев сверху вниз на основе рекуррентной процедуры.
5. Основная задача – выбор переменной для расщепления.
6. Один из наиболее эффективных алгоритмов – C4.5.
7. Один из эффективных методов классификации – метод опорных векторов (SVM).
8. Основа SVM – линейная классификация.
9. Цель SVM – максимизация зазора между опорными гиперплоскостями.
10. Векторы из обучающего множества на граничных плоскостях – опорные.
11. Формальная постановка SVM – задача квадратичного программирования.
12. Основной подход к решению – переход к двойственной задаче.
13. Может применяться для линейно-неразделимой задачи.

Машинное обучение
Лекция 5
Кластеризация данных

Постановка задачи I

Кластерный анализ (кластеризация, таксономия, clustering) –

задача разбиения заданной выборки объектов на непересекающиеся подмножества (кластеры, таксонвы, группы) так, чтобы каждый кластер состоял из схожих объектов, а объекты разных кластеров существенно отличались.

Задача кластеризации – **обучение без учителя** (сравните с задачей классификации).

Входные данные:

- множество данных с описанием объектов.Каждый объект описывается набором характеристик (признаков). Признаки: числовыми / нечисловые.
- метрика (см. далее).

Возможные цели кластеризации:

- понимание данных за счет уменьшения объема (см. социологические опросы, анкетирования и т.д.);
- сжатие данных.

Формальная постановка задачи.

X – множество объектов

Функция расстояния между объектами $\rho(x, x')$.

Обучающая выборка объектов: $X^m = \{x_1, \ldots, x_m\} \subset X$.

Каждому объекту $x_i \in X^m$ должен быть поставлен в соответствие номер кластера y_i.

Множество Y в некоторых случаях известно заранее, однако чаще ставится задача определить оптимальное число кластеров, с точки зрения того или иного критерия качества кластеризации.

Алгоритмизация кластеризации I

Решение задачи кластеризации **принципиально неоднозначно**.

Обобщенный алгоритм кластеризации:

- Отбор выборки объектов для кластеризации.
- Определение множества переменных, по которым будут оцениваться объекты в выборке. При необходимости – нормализация значений переменных.
- Задание метрики.
- Применение метода кластерного анализа для создания кластеров.
- Представление (интерпретация) результатов анализа.

Цель нормализации – чтобы все признаки давали одинаковый вклад при расчете расстояния: в диапазоны $[-1, -1]$ или $[0, 1]$.
Основной принцип: анализируются расстояния между объектами.

Основные метрики:

- Евклидово расстояние:
$$\rho(x, x') = \sqrt{\sum_{i=1}^{n} (x_i - x'_i)^2}.$$
- Квадрат евклидова расстояния (отдаленные объекты – значительный вклад):
$$\rho(x, x') = \sum_{i=1}^{n} (x_i - x'_i)^2.$$
- Манхэттенское расстояние (более робастное, устойчивое к «выбросам»):
$$\rho(x, x') = \sum_{i=1}^{n} |x_i - x'_i|.$$
- Расстояние Чебышева:
$$\rho(x, x') = \max_{i=1}^{n} |x_i - x'_i|.$$

Метрики для оценивания расстояния между кластерами

- Расстояние **ближайшего соседа** (одиночная связь, single-link). Расстояние между двумя наиболее близкими объектами (ближайшими соседями) в различных кластерах (пригодно для кластеров вытянутой формы).
- Расстояние **наиболее удаленных соседей** (полная связь, complete-link). Наибольшее расстояние между любыми двумя объектами в различных кластерах (наиболее удаленными соседями) (не для кластеров вытянутой формы)
- **Невзвешенное попарное среднее**. Среднее расстояние между всеми парами объектов в них.
- **Взвешенное попарное среднее**. При вычислениях размер соответствующих кластеров (число объектов в кластере) используется в качестве весового коэффициента (когда предполагаются неравными размеры кластеров).
- **Невзвешенный центроидный метод**. Расстояние между центрами тяжести.
- **Взвешенный центроидный метод (медиана)**. Используются веса для учета разницы между размерами кластеров (когда неравные размеры кластеров).

Замечание:

обычно при расчете расстояния от кластера C_k до объединения кластеров C_i, C_j в один используется не расчет на основе расстояний между объектами, а расстояние между кластерами $\rho(C_k, C_i)$ и $\rho(C_k, C_j)$.

Способы оценки качества кластеризации:

- внешние (External) – основаны на сравнении результата кластеризации с известным разделением на классы;
- внутренние (Internal) – используется только информация в данных (**наиболее частый** подход).

Внутренние оценки качества кластеризации:

- оценка компактности кластеров (Cluster Cohesion) – внутриклассовое расстояние:
$$WSS = \sum_{i=1}^{m} \rho(x_i, C(x_i)) \to \min,$$
где $C(x_i)$ – ядро (центр) кластера, к которому относится объект x_i;
- оценка отделимости (разделимости) кластеров (Cluster Separation):
$$BSS = \sum_{i=1}^{k} \rho(C(x_i), C) \to \max,$$
где k – количество кластеров; C – центр тяжести объектов;
- коэффициент силуэта (Silhouette Coefficient) – насколько объекты ближе к объектам своего кластера, чем к другим:
$$SC = \max_{j=1}^{k} \tilde{s}_j,$$

$$\tilde{s_j} = \frac{1}{|C_j|} \sum_{x_i \in C_j} s_i.$$

Здесь: близость x_i объектам своего кластера:

$$a_i = \frac{1}{|C(x_i)| - 1} \sum_{j \in C(x_i), i \neq j} \rho(x_i, x_j);$$

среднее отличие x_i от объектов другого кластера C_k, $C_k \neq C(x_i)$:

$$b_{i,k} = \frac{1}{|C_k|} \sum_{j \in C_k} \rho(x_i, x_j);$$

соседний кластер (наименьшее различие, наиболее подходящий кластер для x_i):

$$b_i = \min_{C_k \neq C(x_i)} b_{i,k};$$

силуэт точки:

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}, \quad |C(x_i)| > 1;$$

$$s_i = 0, \quad |C(x_i)| = 1.$$

Здесь $-1 \leqslant s_i \leqslant 1$ ($s_i \to 1$, если $a_i << b_i$).
Чем SC ближе к 1, тем лучше. Центр кластера может определяться как среднее арифметическое (обычно в k-means) или как медианное (медианный силуэт).

Методы кластеризации I

1. Алгоритмы квадратичной ошибки.
Минимизируется функционал:

$$e^2 = \sum_{k=1}^{K} \sum_{i=1}^{n} \|x_i^j - c_j\|^2,$$

где c_j – ядро кластера j (точка со средними значениями характеристик для данного кластера), K – число кластеров.
Самый распространенный алгоритм – **метод k-средних**. Количество кластеров **изначально должно быть задано**.

Алгоритм:

1. Случайно выбирается K точек, являющихся начальными ядрами кластеров.
2. Каждый объект относится к кластеру с ближайшим ядром.
3. Пересчитываются ядра кластеров, исходя из текущего разбиения на кластеры.
4. Если критерий остановки алгоритма не выполнен, возврат на шаг 2.

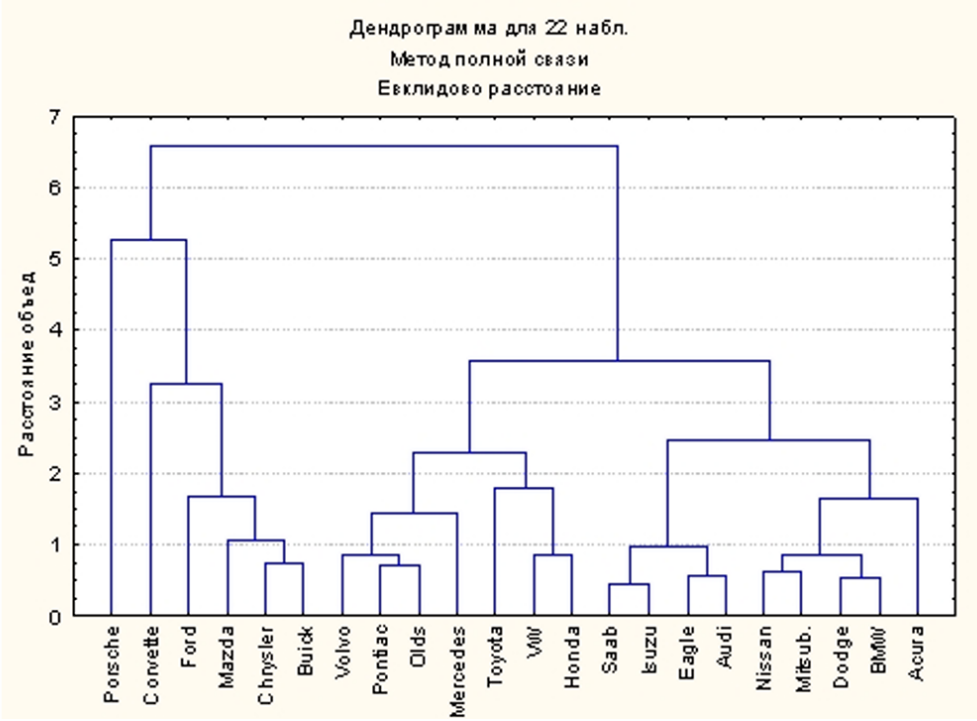
Критерий 1: изменение ошибки меньше некоторого числа ε.
Критерий 2: Если на шаге 2 не было объектов, переместившихся из кластера в кластер.

2. Алгоритмы иерархической кластеризации.
Иногда именно она называется **таксономией**. Крупные кластеры дробятся на более мелкие, те в свою очередь дробятся ещё мельче, и т. д. Результат – иерархическая структура. Каждый объект характеризуется перечислением всех кластеров, которым он принадлежит, обычно от крупного к мелкому.
Пример (классический) – классификация живых существ (Карл Линней, XVIII в.).

Два основных типа:

- **нисходящие** (разделительные, дивизимные, «сверху-вниз»: в начале все объекты помещаются в один кластер, который затем разбивается на все более мелкие кластеры);
- **восходящие** (агломеративные, «снизу-вверх»: сначала каждый объект в отдельном кластере, затем кластеры объединяют во все более крупные).

Визуально – **дендрограмма** (древовидная диаграмма, содержащая m уровней, каждый из которых соответствует одному из шагов процесса последовательного укрупнения кластеров).



Восходящие алгоритмы наиболее популярны. Обычно применяют для оценивания расстояния одиночной связью или полной связью.

Алгоритм:

1. Инициализация. Множество кластеров = множеству объектов. Граф - множество точек – объектов.
2. Пока количество кластеров больше 1:
   - Выбираем 2 объекта x_1 и x_2, расстояние между которыми минимально.
   - Добавляем вершину x_{12}, соединенную с x_1 и x_2.
   - Из множества кластеров удаляем кластеры с x_1 и x_2, добавляем кластер с x_{12}.

Метод Уорда. Исходные значения признаков нормируются. Метод Уорда приводит к образованию кластеров приблизительно равных размеров с минимальной внутрикластерной вариацией. На каждом шаге работы алгоритма объединяются те объекты или кластеры, которые дают **наименьшее приращение** метрики.
Может быть реализован рекурсивно с помощью **алгоритма Ланса-Вильямса (Lance–Williams)**.

Вначале (каждый кластер – объект) – квадрат евклидова расстояния. Предположим, что C_i и C_j – кластеры, предполагаемые к объединению. Все попарные расстояния между кластерами полагаются известными. Рекурсивная формула обновляет значения расстояний между кластерами.

Пусть ρ_{ij}, ρ_{ik} и ρ_{jk} – расстояния между кластерами C_i, C_j и C_k соответственно. Обозначим: $d_{(ij)k}$ – расстояние между новым кластером $C_i \cup C_j$ и C_k. Семейство алгоритмов Ланса-Вильямса:

$$\rho_{(ij)k} = \alpha_i \rho_{ik} + \alpha_j \rho_{jk} + \beta \rho_{ij} + \gamma |\rho_{ik} - \rho_{jk}|,$$

где α_i, α_j, β, γ – параметры, которые могут зависеть от размеров кластеров.
Метод Уорда: n_i, n_j, n_k – размеры соответствующих кластеров.

$$\alpha_l = \frac{n_l + n_k}{n_i + n_j + n_k}, \quad l \in \{i, j\},$$

$$\beta = \frac{-n_k}{n_i + n_j + n_k},$$

$$\gamma = 0.$$

3. DBSCAN. DBSCAN («Пространственная кластеризация приложений с шумом на основе плотности», Density-Based Spatial Clustering of Applications with Noise): поиск областей с высокой плотностью точек и расширение этих областей.

Достоинства DBSCAN:

- устойчивость к выбросам;
- возможность находить кластеры произвольной формы;
- не требуется заранее определять количество кластеров.

Параметры DBSCAN:

- ε – радиус окрестности относительно некоторой точки (чтобы точка считалась соседней);
- *minPts* – минимальное количество точек в ε-окрестности точки, чтобы точка была классифицирована как «основная».

Виды точек в DBSCAN:

- ядерные (core) точки x_i – если в ε-окрестности находится не менее *minPts* точек (включая саму точку);
- прямо достижимые по плотности точки x_j – если $\rho(x_i, x_j) < varepsilon$, где x_i – ядерная точка;
- достижимые точки – если существует путь от ядерной точки x_i до x_j через прямо достижимые точки, каждая из которых (кроме конечной x_j) является основной;
- выбросы – все остальные, которые не достижимы.

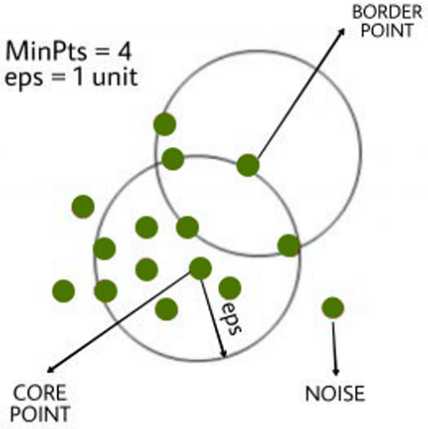


Рис.. Различные виды точек в алгоритме DBSCAN

Обобщенный алгоритм DBSCAN:

1. определяются точки в ε-окрестности каждой точки, выделяются ядерные точки (с не менее чем *minPts* соседями);
2. находятся связные компоненты ядерных точек на графе соседей, игнорируя все неядерные точки;
3. каждой неядерной точке назначается ближайший кластер, если кластер является ε-соседним. В противном случае точка – шум.

Ядерная точка x_i вместе со всеми достижимыми точками – кластер. Точки, не являющиеся ядерными, находятся на границе кластера.

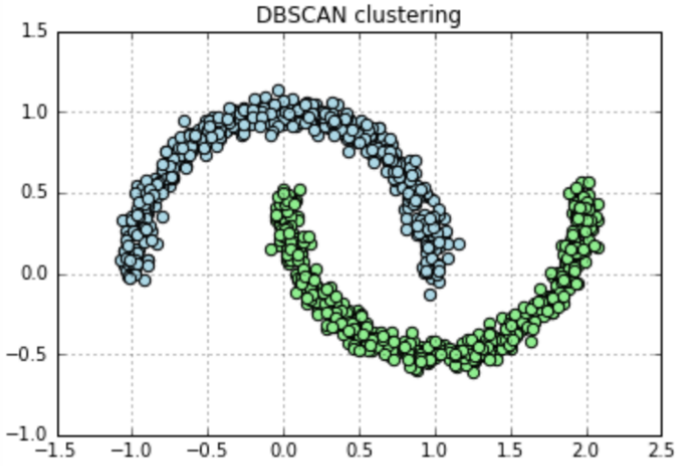


Рис.. Пример результата работы DBSCAN

Определение оптимального количества кластеров I

Определение оптимального числа кластеров.

Варианты:

- Иногда заказчик точно знает, сколько кластеров хочет получить.
- Чаще: в диапазоне от K_1 до K_2.
- Обычно: не знает.

Последний вариант: решение зависит от цели.

Если для машинного использования (снижение объема данных), то K может быть большим.

Если «вручную», то 7 ± 2 (психология, Г. Миллер), большее количество неудобно запоминать и использовать.

Методы для выбора оптимального k:

- **Метод локтя**, основанный на графике суммы квадратов расстояний между объектами и центроидами кластеров (SSE) в зависимости от k. Оптимальным k считается та точка, после которой SSE уменьшается незначительно.
- Метод **силуэта**, основанный на графике среднего значения силуэта для каждого k. Оптимальным k считается точка, где силуэт достигает максимума.
- **Метод комплексной оценки**, основанный на анализе нескольких критериев, таких как динамика перераспределения объектов в кластерах, изменение потенциальной энергии объектов внутри кластеров и характерные точки графиков этих критериев. Оптимальным k считается точка, которая удовлетворяет набору правил, использующих данные критерии.

Метод «локтя» (Elbow method)

Анализируется процент объяснимой вариации, интерпретируемой как функция от числа кластеров:

$$\sum_{i=1}^{K} n_i \frac{\overline{Y}_i - \overline{Y}}{K-1},$$

где $\overline{Y}_i$ – середина i-го кластера, n_i – число объектов в i-ом кластере, $\overline{Y}$ – среднее по всем объектам. Выбирается количество кластеров, которое при добавлении числа кластеров не дает существенного прироста процента объяснимой вариации (Robert L. Thorndike, 1953).

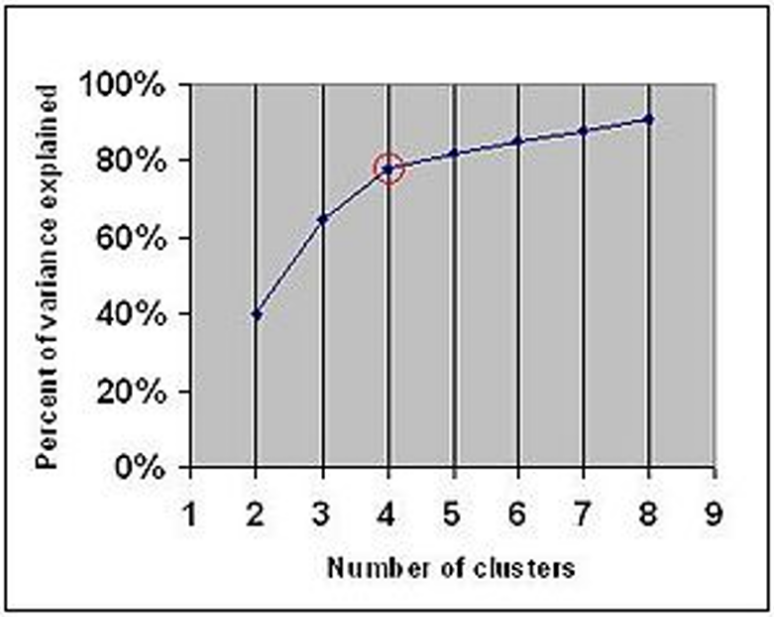


Рис.. Иллюстрация метода локтя

Заключение

1. Задача кластеризации – задача обучения без учителя.
2. Решение задачи кластеризации принципиально неоднозначно.
3. Близость объектов описывается функцией расстояния.
4. Необходимо решение задачи определения оптимального числа кластеров. Одним из методов является метод локтя.

Машинное обучение
Лекция 6
Математическое моделирование

Математическое моделирование. Постановка задачи I

Моделирование –

замещение объекта его моделью с целью получения информации о важнейших свойствах объекта на основе исследования его модели.

Наиболее распространенное – **математическое моделирование**.

Основные предположения при моделировании:

- что является исходными данными (переменными – входными и выходными величинами при явных зависимостях);
- наличие зависимости между исходными данными.

Математическая модель (ММ) –

приближенное описание какого-либо класса явлений реального мира, выраженное с помощью математической символики.



$$\frac{\partial^2 u}{\partial t^2} = a^2 \frac{\partial^2 u}{\partial x^2} + f(x,t)$$

Количественная информация о моделируемой системе определяется числовыми величинами:

- переменными;
- параметрами (обычно на промежутке времени, когда рассматривается функционирование системы, можно считать постоянными).

Классификация математических моделей:

- детерминированные и стохастические;
- статические и динамические;
- дискретные, непрерывные и дискретно-непрерывные;
- сосредоточенные и распределенные;
- линейные и нелинейные;
- стационарные (параметры не меняются со временем) и нестационарные (параметры меняются со временем);
- четкие и нечеткие (основанные на нечеткой логике).

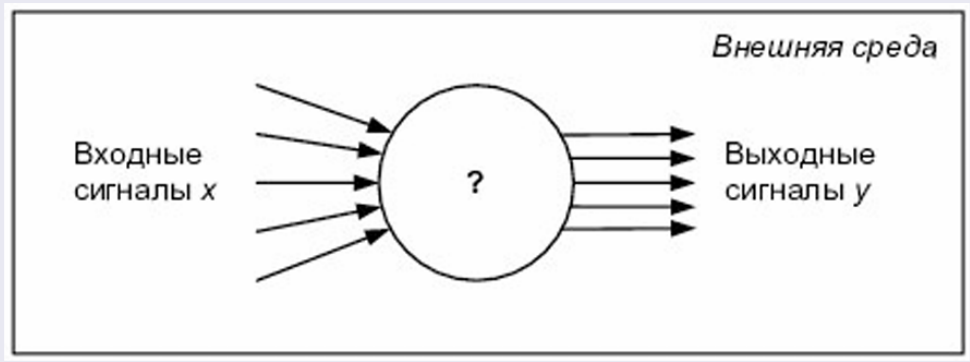
Основные способы построения математических моделей:

- на основе информации о физических и др. принципах функционирования объектов;
- формальное – на основе множества данных (ранее: моделирование систем типа «черный ящик»).

Входы и выходы – переменные модели. В таких случаях поведение «черного ящика» считается зависимым от набора внутренних параметров.

«Черный ящик» (black box) –

система, преобразующая входные сигналы в выходные по неизвестному закону:



Входная информация – обучающее множество:

$$\left[\begin{array}{cccc|cccc} \tilde{x}_{11} & \tilde{x}_{12} & \dots & \tilde{x}_{1n} & \tilde{y}_{11} & \tilde{y}_{12} & \dots & \tilde{y}_{1r} \\ \tilde{x}_{21} & \tilde{x}_{22} & \dots & \tilde{x}_{2n} & \tilde{y}_{21} & \tilde{y}_{22} & \dots & \tilde{y}_{2r} \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ \tilde{x}_{k1} & \tilde{x}_{k2} & \dots & \tilde{x}_{kn} & \tilde{y}_{k1} & \tilde{y}_{k2} & \dots & \tilde{y}_{kr} \end{array}\right]$$

Здесь k – количество примеров, n и r – количество входов и выходов соответственно.
Краткая запись:

$$\langle \tilde{X}, \tilde{Y} \rangle = \{\tilde{x}_i, \tilde{y}_i\}, i = 1, \dots, k,$$

где $\tilde{x}_i \in \mathbb{R}^n$ – входной вектор из i-го примера, $\tilde{y}_i \in \mathbb{R}^r$ – вектор соответствующих указаний учителя.
Строки – **примеры**. Выходы – **указания учителя**. Далее считаем: $r = 1$.
Этапы построения (**идентификации**) формальных математических моделей:

- структурная (определяется многообразие/класс моделей, зависящих от параметров w);
- параметрическая (в терминологии машинного обучения и нейронных сетей: **обучение**).

Линейные по параметрам модели.
Входы:

$$x = \begin{bmatrix} x_1 & x_2 & \dots & x_m \end{bmatrix}^T \in \mathbb{R}^m,$$

параметры:

$$a = \begin{bmatrix} a_1 & a_2 & \dots & a_n \end{bmatrix}^T \in \mathbb{R}^n,$$

выход:

$$y \in \mathbb{R},$$

где $\mathbb{R}$ – множество действительных чисел, m – количество входов, n – количество параметров модели.
ММ – **статическая** (СМ), если значение выхода зависит от входных значений в тот же момент времени. **Явная** СМ (ЯСМ) задается аналитическим выражением, описывающим зависимость выхода от входов и параметров модели в явном виде:

$$y = f(x; a) = f(x_1, x_2, \dots, x_m; a_1, a_2, \dots, a_n).$$

Неявная СМ задается в виде уравнения или системы:

$$g(y, x) = 0.$$

ЯСМ наиболее часто используются для описания систем типа «черный ящик».

Типы явных моделей:

- **линейные по параметрам** модели

$$y = \sum_{i=1}^{n} a_i \varphi_i(x),$$

  где $\varphi_i(x) = \varphi_i(x_1, x_2, \ldots, x_m)$ – базисные функции;
- **нелинейные по параметрам** модели.

Простейшие линейные по параметрам модели:

$$y = \sum_{i=0}^{n} a_i x_i = a_0 + a_1 x_1 + a_2 x_2 + \ldots + a_n x_n.$$

Полиномиальные модели:

$$y = f(x) = a_0 + \sum_{i=1}^{n} a_i x_i + \sum_{i=1}^{n} \sum_{j=1, i \le j}^{n} a_{ij} x_i x_j + \ldots$$

Линеаризуемые модели:

модели, которые с помощью эквивалентных преобразований могут быть приведены к линейным (этот подкласс – очень узкий). Частный случай – **квазилинейные** модели:

$$y = g\left(\sum_{i=0}^{n} a_i \varphi_i(x)\right),$$

где $g(\cdot)$ – некоторая обратимая функция, т.е. $\exists g^{-1}$. Преобразование: замена выходной переменной на $\hat{y} = g^{-1}(y)$.

Пример линеаризуемой модели, которая не является квазилинейной:

$$y = a_1 x^{a_2}, \quad x > 0.$$

Нелинейные по параметрам модели.
Дробно-линейные модели:

$$y = f(x) = \frac{\sum_{i=0}^{n} a_i x_i}{\sum_{i=0}^{n} b_i x_i}.$$

Дробно-полиномиальные модели:

$$y = f(x) = \frac{a_0 + \sum_{i=1}^{n} a_i x_i + \sum_{i=1}^{n} \sum_{j=1, i \le j}^{n} a_{ij} x_i x_j + \ldots}{b_0 + \sum_{i=1}^{n} b_i x_i + \sum_{i=1}^{n} \sum_{j=1, i \le j}^{n} b_{ij} x_i x_j + \ldots}.$$

Линейно-нелинейные по параметрам модели:

модели, которые содержат как нелинейно, так и линейно входящие параметры. Примеры:

- дробно-линейные модели;
- дробно-полиномиальные модели;
- нейронные сети прямого распространения (веса выходного слоя – линейно входящие параметры).

Для их идентификации могут быть применены специальные алгоритмы, основанные на декомпозиции вектора параметров.

Отклонение выхода модели сети y_i на i-м примере и соответствующего указания учителя $\tilde{y}_i$ при текущем входном векторе $x_i \in \mathbb{R}^n$ (**невязка, residual**):

$$\varepsilon_i(w) = y_i(w) - \tilde{y}_i.$$

Вектор невязок:

$$\varepsilon = \varepsilon(w) = y(w) - \tilde{y}.$$

Пусть известные значения вектора параметров $w \in \mathbb{R}^s$. Мгновенный функционал качества для примера i:

$$Q_i = Q_i(w) = Q(\varepsilon_i(w)).$$

Степень соответствия модели данным обучающего множества: интегральный функционал качества обучения:

$$Q = Q(w) = \sum_{i=1}^{k} Q_i(w)$$

Цель обучения:

$$w^* = \arg\min_{w \in \mathbb{R}^s} Q(w).$$

Задача параметрической идентификации – **оптимизационная задача**.

Нормы невязок моделей I

Q – обычно норма вектора невязок ε:

$$Q = \|\varepsilon\|.$$

Наиболее известно семейство норм Гёльдера l_p, которые определяются как

$$\|\varepsilon\|_p = \left(\sum_{i=1}^{k} |\varepsilon_i|^p \right)^{\frac{1}{p}}.$$

Формула может применяться для $0 < p \leqslant \infty$. Наиболее часто используются:

- l_1-норма (робастная, не чувствительная к «выбросам»):

$$\|\varepsilon\|_1 = \left(\sum_{i=1}^{k} |\varepsilon_i|\right),$$

- l_2-норма (традиционная):

$$\|\varepsilon\|_2 = \left(\sum_{i=1}^{k} |\varepsilon_i|^2\right)^{\frac{1}{2}},$$

- l_∞-норма (равномерная, все решения в δ-трубке):

$$\|\varepsilon\|_\infty = \max_{i=1}^{k} |\varepsilon_i|.$$

В конечномерном пространстве все нормы – эквивалентны – существуют $k_1 > 0$ и $k_2 > 0$, т.ч. для любых двух норм $\|\cdot\|^1$ и $\|\cdot\|^2$:

$$k_1\|\cdot\|^1 \leqslant \|\cdot\|^2 \leqslant k_2\|\cdot\|^1.$$

Известные следующие соотношения между нормами:

$$\|\cdot\|_1 \geqslant \|\cdot\|_2 \geqslant \|\cdot\|_\infty.$$

В общем случае, с ростом номера p норма вектора может только уменьшиться:

$$\|\cdot\|_{p_2} \leqslant \|\cdot\|_{p_1} \leqslant k^{\frac{1}{p_1}-\frac{1}{p_2}}\|\cdot\|_{p_2},$$

где $p_2 > p_1 > 0$.

Взвешенные нормы ($v_i > 0$ – веса, отражающие значимость примеров):

- l_1-норма (робастная, не чувствительная к «выбросам»):

$$\|\varepsilon\|_1 = \left(\sum_{i=1}^{k} v_i|\varepsilon_i|\right),$$

- l_2-норма (традиционная):

$$\|\varepsilon\|_2 = \left(\sum_{i=1}^{k} v_i|\varepsilon_i|^2\right)^{\frac{1}{2}},$$

- l_∞-норма (равномерная, все решения в δ-трубке):

$$\|\varepsilon\|_\infty = \max_{i=1}^{k} v_i|\varepsilon_i|.$$

Задание: сравните с подходами оценки расстояния между объектами и кластерами в кластеризации.

Норма, независимо от её вида, необходима для **возможности сравнения** различных вариантов решений задачи параметрической идентификации. Лучший вариант из двух векторов w_1 и w_2 – обеспечивающий меньшее значение нормы векторов невязок: $Q(w_1)$ или $Q(w_2)$.

Замечание:

вектор, обеспечивающий минимум одной нормы, в общем случае может не приводить к минимуму при использовании других норм.

Обозначения функционалов качества (ошибки) в зарубежных работах:

- RMSE (Root Mean Squared Error, квадратный корень из среднего квадратичных ошибок) (редко: Mean Squared Deviation, MSD):

$$RMSE = \sqrt{\frac{1}{k}\sum_{i=1}^{k}|\varepsilon_i|^2},$$

- MAE (Mean Absolute Error, среднее абсолютных ошибок) (также: Mean Absolute Deviation, MAE):

$$MAE = \frac{1}{k}\sum_{i=1}^{k}|\varepsilon_i|.$$

Функционалы MAPE и SMAPE:

- MAPE (Mean Absolute Percentage Error, среднее относительных отклонений):

$$MAPE = \frac{1}{k}\sum_{i=1}^{k}\frac{|\varepsilon_i|}{|\tilde{y}_i|} = \frac{1}{k}\sum_{i=1}^{k}\frac{|y_i - \tilde{y}_|}{|\tilde{y}_i|},$$

  где полагается для всех $\varepsilon_i \neq 0$: $\frac{\varepsilon_i}{0} = \infty$; и $\frac{0}{0} = 1$;
- SMAPE (Symmetric MAPE, симметричное MAPE) (иногда: Adjusted MAPE, скорректированное MAPE):

$$SMAPE = \frac{1}{k}\sum_{i=1}^{k}\frac{|y_i - \tilde{y}_|}{|y_i| + |\tilde{y}_i|}.$$

С практической точки зрения лучше MAPE (SMAPE).

Идентификация моделей в разных нормах I

Идентификация в норме l_2. Это задача о наименьших квадратах (ЗНК).
Достоинство данной нормы -– дифференцируемость на всей области определения при условии дифференцируемости соответствующей модели.
Квадратичный критерий качества может быть записан в следующем виде:

$$Q = Q(w) = \sum_{i=1}^{k}\varepsilon_i^2 = \sum_{i=1}^{k}(\tilde{y}_i - y_i)^2 = \varepsilon^T(w)\varepsilon(w) = \varepsilon^T\varepsilon = \|\varepsilon\|^2, \qquad (1)$$

где $\|\cdot\|$ — символ евклидовой нормы вектора. Задачей параметрической идентификации является нахождение таких значений вектора параметров w, чтобы величина критерия качества минимальной, т.е. $Q(w) \to \min$.
Если $f(x; w)$ линейна по параметрам, то ЗНК называется линейной (ЛЗНК), в противном случае — нелинейной (НЗНК). Метод решения ЗНК называется **методом наименьших квадратов (МНК)**; соответственно типу ЗНК выделяется линейный МНК (ЛМНК) и нелинейный МНК (НМНК).

Линейный метод наименьших квадратов.
Линейная по параметрам модель $y = \sum_{i=1}^{n} w_i \varphi_i(x)$ может быть представлена в виде:

$$y = w_1\varphi_1(x) + w_2\varphi_2(x) + \ldots + w_n\varphi_n(x) = \begin{bmatrix}\varphi_1(x) & \varphi_2(x) & \ldots & \varphi_n(x)\end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}.$$

Из вектора, содержащего базисные функции, на массиве данных составим матрицу:

$$\Phi = \begin{bmatrix} \varphi_1(x_{11}) & \varphi_2(x_{12}) & \ldots & \varphi_n(x_{1n}) \\ \varphi_1(x_{21}) & \varphi_2(x_{22}) & \ldots & \varphi_n(x_{2n}) \\ \vdots & \vdots & \ddots & \vdots \\ \varphi_1(x_{k1}) & \varphi_2(x_{k2}) & \ldots & \varphi_n(x_{kn}) \end{bmatrix} \in \mathbb{R}^{k\times n}.$$

Вектор выходов модели на массиве входных данных $\tilde{y}$ получается следующим образом:

$$\tilde{y} = \begin{bmatrix} \tilde{y}_1 \\ \tilde{y}_2 \\ \vdots \\ \tilde{y}_k \end{bmatrix} = \Phi w \in \mathbb{R}^k.$$

Обозначим вектор выходных значений из таблицы как

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_k \end{bmatrix} \in \mathbb{R}^k,$$

что не следует путать с обозначением выходной переменной y. Тогда $\varepsilon = \tilde{y} - y = \Phi w - y$ и ЗНК может быть записана как

$$Q(w) = \|\Phi w - y\|^2 \to \min. \tag{2}$$

Оптимальные значения вектора параметров определяется по формуле

$$w^* = \Phi^+ y,$$

где $\Phi^+ \in \mathbb{R}^{n\times k}$ — **псевдообратная** к $\Phi \in \mathbb{R}^{k\times n}$ матрица.
Это соотношение позволяет решить ЛЗНК за один шаг.

Условия Мура-Пенроуза для псевдообратной матрицы A^+:

$$AA^+A = A;$$
$$A^+AA^+ = A^+;$$
$$(AA^+)^T = AA^+;$$
$$(A^+A)^T = A^+A.$$

Свойства A^+:

1. существует для любой матрицы;
2. единственна;
3. для квадратной невырожденной матрицы $A^+ = A^{-1}$;
4. если A – матрица полного столбцового ранга, то

$$A^+ = (A^TA)^{-1}A^T.$$

Алгоритмы вычисления A^+:

1. Гревиля;
2. Фаддева;
3. на основе QR-разложения;
4. на основе SVD-разложения.

Нелинейный метод наименьших квадратов.

НМНК основан на итерационном способе нахождения решения: строится такая последовательность векторов w^i, что:

$$Q(w^0) > Q(w^1) > \ldots > Q(w^{s-1}) > Q(w^s).$$

Вычисления прекращаются на s-м шаге итерационного процесса, когда выполнено условие останова, например, $|Q(w^{s-1}) - Q(w^s)| < \varepsilon$, где ε — заданная точность. Кроме точности первоначально задается стартовый вектор w^0; разные стартовые вектора могут привести к разным локальным минимумам. (При решении НЗНК нет гарантий того, что последовательность w^i приведет к глобальному минимуму функции 1.)

Линейная модель $R(w)$ в окрестности текущего вектора w^i представляется как

$$R(w) \approx R(w^i) + J(w^i)(w - w^i),$$

где $J(w^i) = R'(w^i) \in \mathbb{R}^{k \times n}$ — матрица Якоби вектор-функции от векторного аргумента. Матрицу Якоби составляют строки — транспонированные градиенты элементов $\varepsilon_i(w)$ по векторному аргументу w:

$$J(w) = \begin{bmatrix} \nabla^T \varepsilon_1(w) \\ \nabla^T \varepsilon_2(w) \\ \vdots \\ \nabla^T \varepsilon_k(w) \end{bmatrix} = \begin{bmatrix} \frac{\partial \varepsilon_1}{\partial w_1} & \frac{\partial \varepsilon_1}{\partial w_2} & \cdots & \frac{\partial \varepsilon_1}{\partial w_n} \\ \frac{\partial \varepsilon_2}{\partial w_1} & \frac{\partial \varepsilon_2}{\partial w_2} & \cdots & \frac{\partial \varepsilon_2}{\partial w_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \varepsilon_k}{\partial w_1} & \frac{\partial \varepsilon_k}{\partial w_2} & \cdots & \frac{\partial \varepsilon_k}{\partial w_n} \end{bmatrix} = \begin{bmatrix} \frac{\partial y_1}{\partial w_1} & \frac{\partial y_1}{\partial w_2} & \cdots & \frac{\partial y_1}{\partial w_n} \\ \frac{\partial y_2}{\partial w_1} & \frac{\partial y_2}{\partial w_2} & \cdots & \frac{\partial y_2}{\partial w_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_k}{\partial w_1} & \frac{\partial y_k}{\partial w_2} & \cdots & \frac{\partial y_k}{\partial w_n} \end{bmatrix}. \tag{3}$$

Отсюда (исходя из $R(w) = 0$) следует, что вектор w^{i+1} определяется на основе формулы

$$w^{i+1} = w^i - J(w^i)^+ R(w^i). \tag{4}$$

Алгоритм пересчета 4 – **метод Гаусса-Ньютона с псевдообращением для НЗНК** (по аналогии с методом Ньютона решения нелинейных уравнений).
Значительно чаще используется **демпфированный** методом Гаусса-Ньютона с псевдообращением для НЗНК:

$$w^{i+1} = w^i - \lambda J(w^i)^+ R(w^i), \tag{5}$$

где $\lambda \in \mathbb{R}$, $\lambda > 0$ — длина шага. Демпфированный метод Гаусса-Ньютона гарантирует выполнение условия $Q(w^{i+1}) < Q(w^i)$ при стремлении длины шага к нулю: $\lambda \to 0$.

Оптимальное значение λ определяется с помощью любого из методов одномерной оптимизации, простейшим из которых является метод дробления шага:

1. задается $\lambda = 1$;
2. если $\lambda < \delta$, где δ — минимально возможная длина шага, то решение НЗНК считается известным, процесс поиска w^{i+1} прекращается;
3. рассчитывается значение w^{i+1} по формуле 5 при текущем значении λ;
4. если $J(w^{i+1}) < J(w^i)$, то значение w^{i+1} считается определенным и поиск прекращается, иначе полагают $\lambda := \frac{\lambda}{c}$ ($c > 1$ — делитель шага — некоторая заданная константа) и происходит возврат на шаг 2.

Обычно берется $c = 2$.

Сложность применения методов данного типа:

при нелинейной функции функционал качества имеет многоэкстремальный характер. Решение: технологии мультистарта, общие методы глобальной оптимизации (обычно не учитывают особенность задачи оптимизации в норме l_2).

Сложность применения методов данного типа: при нелинейной функции функционал качества имеет многоэкстремальный характер. Решение: технологии мультистарта, общие методы глобальной оптимизации (обычно не учитывают особенность задачи оптимизации в норме l_2).

Идентификация в норме l_1. Это задача о наименьших модулях (задача о наименьших абсолютных отклонениях).
Отличия задач идентификации в l_1 и l_2 для линейных по параметрам моделей:

- решение задачи о наименьших модулях более робастно по отношению к входным данным (менее чувствительно к выбросам);
- несколько решений задачи о наименьших модулях при единственном решении задачи о наименьших квадратах.

Линейная модель в l_1 – задача линейного программирования (метод внутренней точки и др.).
MAE – по сути, задача о наименьших модулях, известная также как **медианная регрессия**. Это частный случай **квантильной регрессии** (quantile regression).

Справка из математической статистики

Пусть X – случайная величина, $F_X(x) = p\{X < x\}$ – ее функция распределения. α-квантиль или квантиль уровня α, $0 < \alpha < 1$, – число $x_\alpha \in \mathbb{R}$:

$$p\{X < x_\alpha\} \leqslant \alpha,$$

$$p\{X \geqslant x_\alpha\} \leqslant 1 - \alpha.$$

Для непрерывного распределения:

$$F_X(x_\alpha) = \alpha.$$

- $0,25$-квантиль – первый (нижний) квартиль;
- $0,5$-квантиль – медиана или второй квартиль;
- $0,75$-квантиль – третий (верхний) квартиль.

Квантильная регрессия – минимизация функционала

$$Q(w;\alpha) = \frac{1}{k}\left[\alpha \sum_{i:\tilde{y}_i \geqslant y_i} |\tilde{y}_i - y_i| + (1-\alpha) \sum_{i:\tilde{y}_i < y_i} |\tilde{y}_i - y_i|\right].$$

Если модель – линейная, то задача квантильной регрессии может быть сведена к задаче линейного программирования. Обозначим:

$$e_i = \tilde{y}_i - y_i,$$

$$u_i^+ = max\{e_i, 0\}, \quad u_i^- = max\{-e_i, 0\}, \quad e_i = u_i^+ - u_i^-.$$

Тогда минимизируется функция

$$\sum_{i=1}^{k} [\alpha u_i^+ + (1-\alpha) u_i^-]$$

с ограничениями

$$u_i^+ \geqslant 0, \quad u_i^- \geqslant 0, \quad \tilde{y}_i - y_i = u_i^+ - u_i^-.$$

Задача MAPE –

взвешенная медианная регрессия (квантильная регрессия с $\alpha = 0,5$) с весами $\frac{1}{\tilde{y}_i}$.

Идентификация в норме l_1. Нелинейная модель.
Нелинейная модель: обе задачи – многоэкстремальный характер, характерно только первое отличие. Подход к решению – **итерационно перевзвешиваемый метод наименьших квадратов** (ИПМНК, **Iteratively Reweighted Least Squares**, IRLS). Последовательно, при разных весах v_i, решаются задачи минимизации

$$\|\varepsilon\|_2 = \sum_{i=1}^{k} v_i(w)\varepsilon_i^2,$$

ИПМНК реализует пересчёт вектора параметров по формуле

$$w^{(t+1)} = \arg\min_{w} \sum_{i=1}^{k} v_i(w^{(t)})\varepsilon_i^2.$$

Идея – чем дальше точка, тем с меньшим весом она учитывается.

Веса задачи на начальном шаге принимаются равными 1 (обычная ЗНК), затем пересчитываются по формуле

$$v_i(w^{(t)}) = \frac{1}{|y_i(w^{(t)}) - \tilde{y}_i|},$$

где в случае равенства знаменателя нулю в качестве веса берётся некоторое достаточно большое положительное число, например 10^4. Для каждого шага при пересчёте вектора параметров необходимо решать нелинейную ЗНК. Таким образом, решение задачи оптимизации в l_1 весьма ресурсоёмко по сравнению с оптимизацией в l_2.

Идентификация в норме l_p. ИПМНК может быть применен для решения задачи не только при $p = 1$, но и при других $p < \infty$. Он применим даже для $0 < p < 1$. Отличие – в способе пересчёта весов:

$$v_i(w^{(t)}) = |y_i(w^{(t)}) - \tilde{y}_i|^{p-2}.$$

Как видно, для $p = 2$: обычный МНК.
Таким образом, ИПМНК можно считать **универсальным подходом** для решения задач параметрической идентификации при минимизации различных норм l_p, в основе которого лежит **МНК**.

Идентификация в норме l_∞. Эта задача минимаксная.
Если модели – дифференцируемые, то минимизируемая функция в общем случае не является дифференцируемой, она бу-дет кусочно-гладкой. Следствие: для ее решения методы оптимизации гладких функций в традиционном виде применяться не могут.

Для решения этой задачи может использоваться подход, состоящий в линеаризации исходной задачи и приведению её к виду задачи линейного программирования. Такой подход – SLP (Sequential Linear Programming, последовательное линейное программирование).

Для нелинейных моделей могут эффективно применяться методы глобальной оптимизации на основе методов **интервального анализа**. Их особенность: они гарантируют глобальность найденного решения даже для многоэкстремальных задач.

Основой многих интервальных методов является оценка прообраза функции для заданной области значений $[Q_{min}, Q_{max}]$. Базовый эффективный алгоритм – SIVIA (Set Inverter Via Interval Analysis, обращение множества посредством интервального анализа). При использовании этого алгоритма существенным оказывается учёт времени расчёта функционала Q. Расчёт часто существенно сокращается, когда значение функции при некотором номере строки данных i^*, будет больше нужного множества значений, то есть когда $|y_i(w^{(t)}) - \tilde{y}_i| > Q_{max}$.

Заключение

1. Центральный этап анализа данных – математическое моделирование.
2. Математические модели делятся на два больших класса – линейные и нелинейные по параметрам модели.
3. Построение математических моделей состоит из двух основных этапов – структурной и параметрической идентификации.
4. Формализация задачи параметрической идентификации – задача оптимизации.
5. Близость модели к описываемым данным формализуется через понятие нормы вектора невязок.
6. Обычно в качестве нормы вектора невязок выступают нормы в пространствах l_1, l_2, l_∞.
7. Для минимизации нормы в каждом пространстве требуется применение соответствующих методов оптимизации.
8. Универсальным методом решения задач во многих нормах l_p является итерационный перевзвешиваемый метод наименьших квадратов (IRLS).
9. Для минимизации равномерной нормы перспективными являются методы интервального анализа.
10. Важной нормой при моделировании является норма MAPE. В линейном случае это взвешенная задача медианной регрессии в норме l_1.

Машинное обучение
Лекция 7
Оценка и повышение качества математических моделей

Оценка качества моделей I

Оценка качества моделей (в данной лекции) =

оценка **адекватности** моделей – проверка соответствия модели **реальной** системе.
Отличие от:

- точности моделей;
- качества с точки зрения бизнес-метрики.

Статистические процедуры оценки адекватности –

проверка предположения, что модельные выходы $y(x)$ отличаются от реальных y не более, чем на ε:

$$|y(x) - y| \leqslant \varepsilon.$$

Вместо реальных y рассматриваются результаты эксперимента (известные) $\tilde{y}_i$ и модельные выходы $y_i(x)$ – совокупность с.в.

$$\varepsilon_i = |y_i(x) - \tilde{y}_i|, \quad i = 1, \ldots, k.$$

Можно ли те i, для которых $\varepsilon_i > \varepsilon$, объяснить случайными причинами, или их наличие существенно?
Гипотеза об адекватности модели действительности (H_0): совокупность ε_i не дает оснований отказаться от модели. H_1 – гипотеза, что модель отвергается.

Ошибка первого рода –

принимается H_1 при верной H_0. Вероятность этой ошибки α – риск разработчика.

Ошибка второго рода –

принимается H_0 при верной H_1. Вероятность этой ошибки β – риск потребителя.

Критерии оценки гипотез об адекватности:

1. Критерий согласия χ^2 Пирсона (анализ количества совпадений и отклонений).
2. Критерий Смирнова-Колмогорова (анализ максимального отклонения).
3. Критерий Фишера (анализ дисперсий).

Коэффициент детерминации

$$R^2 = 1 - \frac{\sum_{i=1}^{k}(y(x_i) - \tilde{y}_i)^2}{\sum_{i=1}^{k}(\tilde{y}_i - \bar{\tilde{y}})^2}.$$

Измеряет долю дисперсии, объясненную моделью, в общей дисперсии целевой переменной. Фактически – нормированная среднеквадратичная ошибка (замена MSE).
$R^2 \to 1$ – модель хорошо объясняет данные.
$R^2 \to 0$ – прогнозы сопоставимы с константным прогнозированием.

Методы оценки адекватности в машинном обучении:

- оценка модели на тестовом множестве;
- метод скользящего экзамена;
- кросс-валидация.

Оценка на тестовом множестве:

применяется, если количество примеров k – большое число.
Исходное множество данных $\{X, Y\}$ разбивается на два:

- обучающее ($70 - 80\%$, обычно 80%);
- тестовое ($30 - 20\%$, обычно 20%). Иногда называется **holdout** (придержанным) множеством данных.

Способы отбора примеров:

- случайным образом (p/p);
- стратифицированная выборка – когда данные можно разделить на непересекающиеся подгруппы – страты (для каждой страты – выборка случайно на основе p/p).

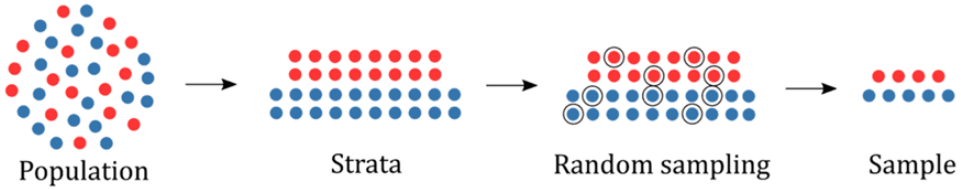


Рис. 1. Иллюстрация стратифицированной выборки

Замечание:

иногда из обучающего (!) множества выделяется **проверочное** (**валидационное**) (validation) множество (иногда – developmetnt set, dev set) для:

- настройки гиперпараметров;
- раннего останова.

Путаница: иногда – обратное использование терминов «проверочное» и «тестовое».
Важно (!): для итоговой цоенки адекватности используется множество, никак не используемое в обучении.

Метод скользящего экзамена:

применяется, если количество примеров k – мало.
Пусть $\{X, Y\} = \{\tilde{x}_i, \tilde{y}_i\}$, $i = 1, \ldots, k$, – исходное множество.
Обозначим: $\{X, Y\}_i$ – обучающее множество без i-го примера.
Замечание: иногда называется методом **джекнайф**.

Идентификация моделей на основе **выбора с возвращением** из обучающего множества последовательно для всех $i = 1, \ldots, k$.
Оценка адекватности – на основе извлеченного примера i.
Преимущества:

- используется вся исходная выборка для параметрической идентификации;
- используется полностью формализованная ошибки обобщения, не требующая эмпирического отбора тестовых пар;
- начальная генерация значений параметров модели может производиться в самом начале, один раз, для всех обучающих множеств.

Развитие – групповой скользящий экзамен:

каждый раз удаляется p примеров; всего получается C_k^p процедур идентификации.
Замечание: иногда называется **групповым методом джекнайф**.

Кросс-валидация (перекрёстная проверка) k-fold:

1. Обучающая выборка разбивается на k непересекающихся одинаковых по объему частей;
2. Производится k итераций:
   - модель обучается на $k - 1$ части обучающей выборки;
   - модель тестируется на части обучающей выборки, которая не участвовала в обучении.

Каждая из k частей используется для тестирования один раз.
Обычно: $k = 10$ (для небольшого множества: 5 в случае малого размера выборки).

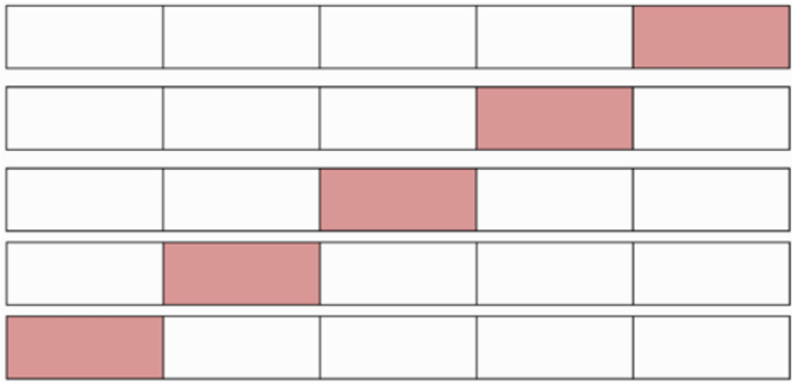


Рис. 2. Иллюстрация k-fold кросс-проверки

Регуляризация в построении математических моделей I

Слишком большие значения параметров **могут увеличивать неустойчивость модели** по отношению ко входным данным.
Это приводит к **переобучению**, **снижает адекватность** модели.

Главная причина роста значений –

высокая корреляция входных величин.
Если $corr(x_1, x_2) \approx 1$, модели

$$y = 1 * x_1 \approx 2 * x_1 - 1 * x_2 \approx 1000 * x_1 - 999 * x_2 \approx \dots,$$

т.е. значения параметров могут безгранично возрастать.

Регуляризация А.Н. Тихонова (1965 г.) –

метод приближённого решения **некорректно поставленных** операторных задач.

Пример ЛЗНК.

$$Q = \|\Phi w - y\|_2^2 \to \min$$

Нормальное псевдорешение является **минимальным** в 2-норме среди всех псевдорешений, то есть фактически, оно решает задачу минимизации функционала

$$\|\Phi w - y\|_2^2 + \lambda \|w\|_2^2 \to \min,$$

где $\lambda > 0$ – некоторый множитель.
Сформулированная задача называется также гребневой регрессией.

Гребневая (Ridge) регрессия:

$$Q + \lambda \|w\|_2^2 \to \min.$$

Аналогично можно

LASSO (Least Absolute Selection and Shrinkage Operator):

$$Q + \lambda \|w\|_1 \to \min.$$

Здесь $\lambda > 0$ – **параметр регуляризации**.

Возможное применение:

при количестве параметров > количества примеров (см. глубокие НС).

Обобщение для пары произвольных норм l_p и l_q:

В общем случае для нелинейных по параметрам моделей задача параметрической идентификации запишется в виде:

$$Q = \|r\|_p^p + \lambda \|w\|_q^q,$$

что может быть представлено в виде задачи условной минимизации функции $\|r\|_p^p$ при условии $\|w\|_q^q \leqslant t$, где $t \geqslant 0$ – настраиваемый параметр, который обычно берётся равным $\frac{1}{\lambda}$.
Чем меньше значение t, тем более значимым является член регуляризации.

Случай $p = q = \infty$:

$$Q = \max_{i=1}^{k} |f(x_i, w) - y_i| + \lambda \max_{i=1}^{m} |w_i|.$$

При использовании алгоритмов идентификации на основе методов интервального анализа более предпочтительно представление в виде задачи условной минимизации функции

$$\max_{i=1}^{k} |f(x_i, w) - y_i|$$

при условии

$$\max_{i=1}^{m} |w_i| \leqslant t,$$

что означает необходимость минимизации на брусе одинакового размера по каждой координате:

$$w_i \in [-t; t] \quad \forall i = 1, \ldots, m.$$

Примечание:

в регуляризационном слагаемом можно учитывать не все, а **часть** параметров.

Гибридизация видов регуляризации

L_1- и L_2-регуляризация вектора параметров (L_{12}) – гибридный метод регуляризации, который сочетает в себе возможности L_1- и L_2-регуляризации для предотвращения переобучения модели. Используется дополнительный параметр α, с помощью которого можно управлять сочетанием L_1- и L_2-регуляризаций.

$$L_{12} = \lambda \left[\alpha \sum_{j=1}^{m} |w_j| + (1 - \alpha) \sum_{j=1}^{m} |w_j|^2 \right],$$

где $0 \leqslant \alpha \leqslant 1$.
Пример в Python: **ElasticNet** (sklearn).

Обобщенная регуляризация:

несколько регуляризуемых параметров в разных нормах:

$$Q = \|r\|_p^p + \lambda \sum_{j=1}^{l} \alpha_j \|w\|_l^l,$$

где $0 \leqslant \alpha_i \leqslant 1, \quad j = 1, \ldots, l$, $\sum_{j=1}^{l} \alpha_j = 1$ – множество параметров, задающих значимость регуляризации в разных нормах, а λ – общая значимость регуляризации.

Косвенные методы регуляризации:

- dropout (метод отсева, метод исключения): в процессе идентификации некоторые параметры случайным образом отключаются, их значения не корректируются;
- ранний останов (использование валидационного множества): сохраняются значения параметров, пока ошибка на валидационном множестве уменьшается, при возрастании возвращаются к предыдущим параметрам;
- аугментация (искусственное увеличение объёма) данных.

Понятие и определение оптимальных значений гиперпараметров I

Модели (алгоритмы) машинного обучения имеют набор **гиперпараметров** (аргументов), которые должны быть заданы специалистом.
Библиотека Scikit-Learn в Python имеет набор гиперпараметров по умолчанию, которые достаточно хорошо работают для всех моделей, но они не обязательно являются наилучшими для каждой из проблем.

Основной принцип оптимизации гиперпараметров –

метод проб и ошибок.

Техники, используемые для оптимизации гиперпараметров:

1. сеточный поиск;
2. случайный поиск.

Сеточный поиск

- Определяется область или сетка параметров, в которую включается набор возможных значений гиперпараметров, которые могут быть использованы для построения модели.
- Используется метод поиска по сетке для размещения этих гиперпараметров в матрице, и модель обучается на каждой комбинации значений гиперпараметров.
- Выбирается модель с наилучшими показателями качества модели.

Важно:

в GridSearchCV используется оценка адекватности на основе k-fold кросс-валидации, а минимизироваться может **другой функционал качества!**

Случайный поиск

В то время как поиск по сетке рассматривает все возможные комбинации гиперпараметров, чтобы найти лучшую модель, случайный поиск выбирает и тестирует только случайную комбинацию гиперпараметров.
Эта техника вместо исчерпывающего поиска делает случайную выборку из сетки гиперпараметров.
Указывается общее количество попыток случайного поиска, прежде чем будет получена наилучшая модель.

Сравнение подходов

- Используется поиск по сетке, если есть примерный диапазон известных значений гиперпараметров, которые будут работать хорошо. Следует убедиться, что область значений параметров невелика, так как поиск по сетке может занять очень много времени.
- Случайный поиск лучше применять в широком диапазоне значений, если еще нет представления о параметрах, которые будут хорошо работать в модели. Случайный поиск быстрее, чем поиск по сетке, и его всегда следует использовать, когда большая область значений параметров.

Вывод относительно поиска гиперпараметров:

При больших размерах области значений параметров можно сначала использовать случайный поиск, так как он быстрее. Затем, используя лучшие гиперпараметры, найденные при случайном поиске, сузить сетку параметров и ввести в сеточный поиск меньший диапазон значений.

Ансамблевые методы моделирования I

Ансамблевые методы (Ensemble methods) –

набор простых моделей $f_i(x), i = 1, \ldots, p$ вместо одной сложной (публикации с 1988 г.). Результаты лучше, чем те, которые получены на одной простой модели.

Плюсы ансамблирования:

снижение смещения и дисперсии при оценивании.

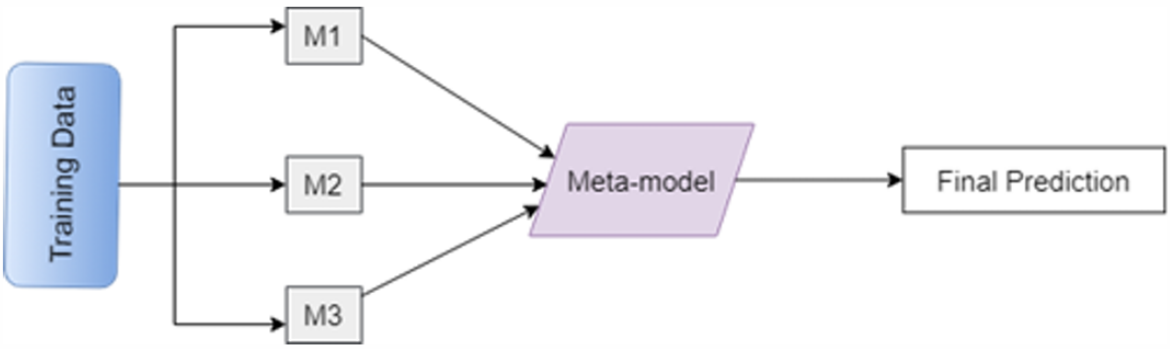
Подходы к ансамблированию:

- **стекинг** (stacking, иногда – stacked generalization) – построение нескольких (возможно, разных) моделей на множестве (или его части), дальнейшее обучение мета-алгоритма для преобразования выходов базовых моделей в реальные выходы;
- **бустинг** (boosting) – последовательное построение моделей, каждая пытается снизить оставшуюся ошибку после построения предыдущих моделей;
- **бэггинг** (bagging, bootstrap aggregating) – обучение моделей на случайных подвыборках из $\{X, Y\}$ с возвращением.

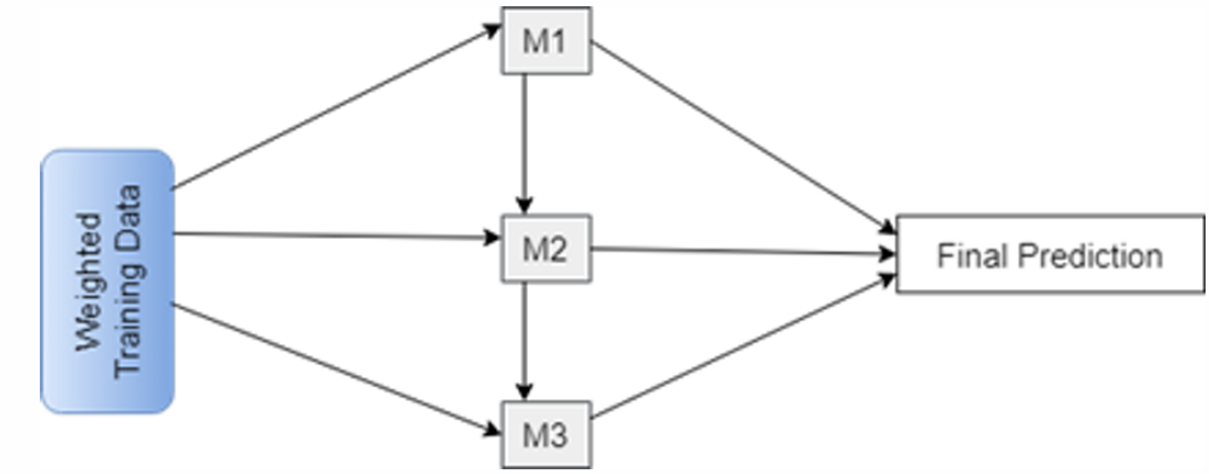
Результат – взвешенная комбинация (в частности, $\alpha_i = 1/p$):

$$f(x) = \sum_{i=1}^{p} \alpha_i f_i(x), \quad \alpha_i > 0, \quad \sum_{i=1}^{p} \alpha_i = 1.$$

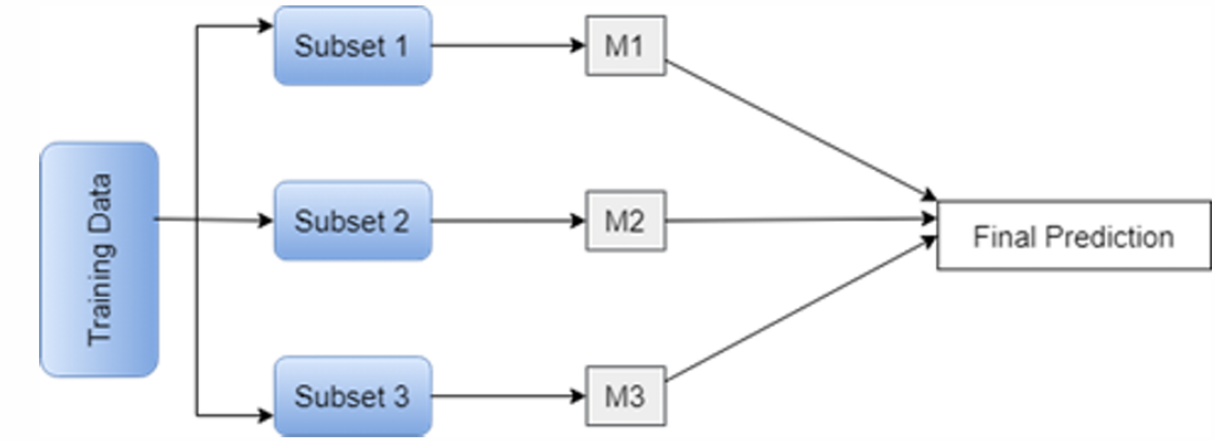
Стекинг



Бустинг



Бэггинг



ЗАКЛЮЧЕНИЕ (ОСНОВНЫЕ МОМЕНТЫ ЛЕКЦИИ)

1. Для оценки качества моделей можно использовать тестовое множество, метод скользящего экзамена и кросс-валидацию.
2. Регуляризация – это способ повышения степени адекватности моделей за счет ограничения значений параметров.
3. При построении моделей может потребоваться определение оптимальных значений аргументов математических методов и алгоритмов — это может быть произведено путем случайного или сеточного поиска в пространстве гиперпараметров.
4. Повышение качества построения моделей может быть достигнуто путем использования ансамблей более простых моделей вместо использования сложных нелинейных математических моделей.

Машинное обучение
Лекция 8
Нейросетевое моделирование

Основы теории нейронных сетей (НС) I

- 1943 г. – МакКаллок (McCulloc) и Питтс (Pitts) «Логическое исчисление идей, относящихся к нервной деятельности»
- 1949 г. – Хебб (Hebb) «Организация поведения»;
- 1957 г. – Розеблатт – первый компьютер-нейронная сеть – перцептрон – обработка логических задач – линейная классификация
- 1969 г. – Минский (Minsky) и Пейперт (Papert) «Перцептроны»: показана ограниченность возможностей НС (XOR-problem);
- 1982 г. – Хопфилд (Hopfield): работы по математическим основам динамики НС;
- 1984 г. – Кохонен (Kohonen): сети, обучающиеся без учителя;
- 1986 г. – Румельхарт (Rumelhart) и МакКлеланд (McCleland): алгоритм обратного распространения ошибки для обучения многослойных НС;
- 1989 г. – ЛеКун (LeCun) - сверточные нейронные сети, распознавание рукописных индексов на конвертах (zip-кодов);
- 1998 г. – ЛеКун (LeCun) – распознавание чеков;
- 2006 г. – Хинтон (Hinton) – алгоритм обучения глубоких слоев в сетях – «умный» выбор начальных значений весов;
- 2010 г. – Глорот (Glorot) и Бенджио (Bengio) – анализ сложности обучения глубоких НС, критика дифференцируемых функций активации;
- 2012 г. – Хинтон (Hinton) – глубокие сверточные НС (deep convolutional NN) - отличные результаты в распознавании образов (ImageNet) – 15,3 % ошибки (26,2 % - ближайший конкурент).

Нейросетевое моделирование – математическое моделирование с использованием моделей специального вида (нейронных сетей).
Искусственный нейрон: вход $x \in \mathbb{R}^n$, выход $y \in \mathbb{R}$.

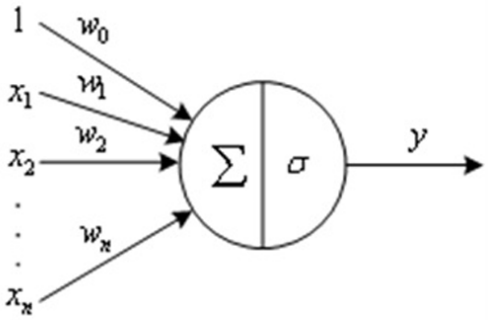


Рис.. (Искусственный) нейрон

Вычисление выхода:

1. уровень активации (активности) нейрона *net*:

$$net = w_0 + \sum_{i=1}^{n} w_i x_i, \tag{1}$$

где $\begin{bmatrix} w_0 & w_1 & \dots & w_n \end{bmatrix}^T = w \in \mathbb{R}^{n+1}$ – вектор весов (весовых коэффициентов, параметров) нейрона;

2. функция активации (ФА):

$$y = \sigma(net),$$

где $\sigma(\cdot) : \mathbb{R} \to \mathbb{R}$ – некоторая, как правило, нелинейная функция.

Основные ФА $\sigma(\cdot)$:

- пороговая (рис. 3)

$$\sigma(net) = \begin{cases} 1, & net \geqslant T, \\ 0, & net < T, \end{cases}$$

где T – значение порога (threshold level);

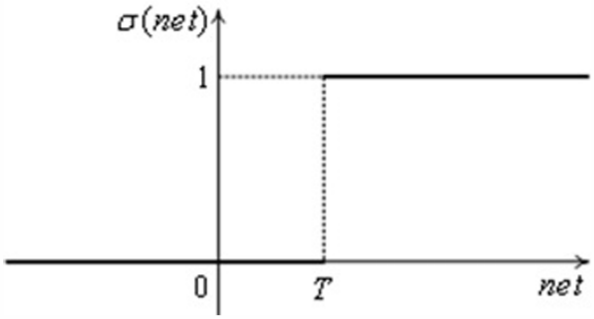


Рис.. Пороговая функция

- линейная пороговая (рис. 4)

$$\sigma(net) = \begin{cases} 1, & net \geqslant T_2, \\ \dfrac{net - T_1}{T_2 - T_1}, & T_1 < net < T_2, \\ 0, & net \leqslant T_1, \end{cases}$$

где T_1 и T_2 – пороговые константы;

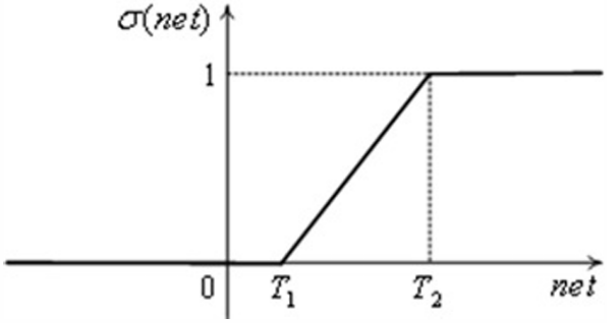


Рис.. Линейная пороговая функция

- гиперболический тангенс (сигмоидная биполярная) (рис. 5)

$$\sigma(net) = th(net) = \frac{e^{net} - e^{-net}}{e^{net} + e^{-net}};$$

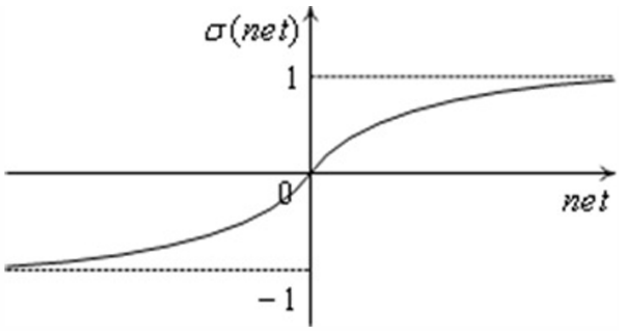


Рис.. Гиперболический тангенс

- сигмоидная логистическая (сигмоидная униполярная) (рис. 6)

$$\sigma(net) = \frac{1}{1 + e^{-net}}.$$

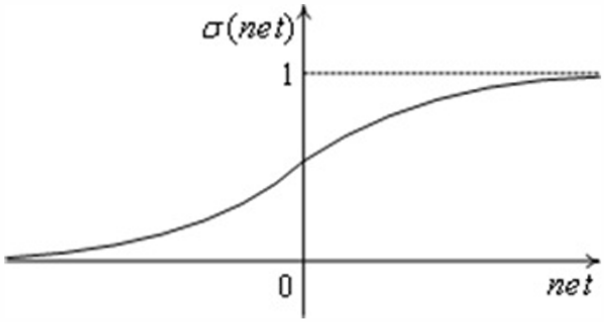


Рис.. Сигмоидная логистическая функция

Перцептрон. Классификация. XOR-проблема I

Перцептрон*: ФА – пороговая

$$\sigma(net) = \begin{cases} 1, & w_1x_1 + w_2x_2 + \ldots + w_nx_n \geqslant T, \\ 0, & w_1x_1 + w_2x_2 + \ldots + w_nx_n < T \end{cases}$$

Пример. Булева функция.

$$y = f(x_1, x_2) = x_1 \vee x_2$$

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

Задача: определить веса перцептрона w_1, w_2, порог T, чтобы выдавать правильный ответ для всех входных значений.

$$\sigma(net) = \begin{cases} 1, & w_1x_1 + w_2x_2 \geqslant T, \\ 0, & w_1x_1 + w_2x_2 < T \end{cases}$$

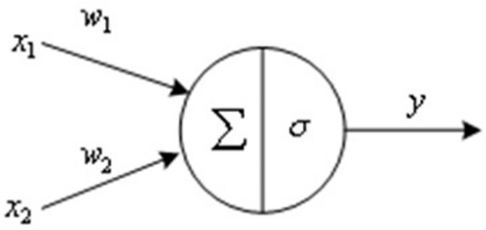


Рис.. Перцептрон

Бесконечное множество решений.

$$w_1 = w_2 = 0,5; \quad T = 0,4$$

Пусть $x_1 = 1$, $x_2 = 0$:

$$net = 0,5 \cdot 1 + 0,5 \cdot 0 = 0,5$$

$$net = 0,5 \geqslant T = 0,4$$

$$y = 1$$

Две полуплоскости – граница:

$$0,5x_1 + 0,5x_2 = 0,4.2.$$

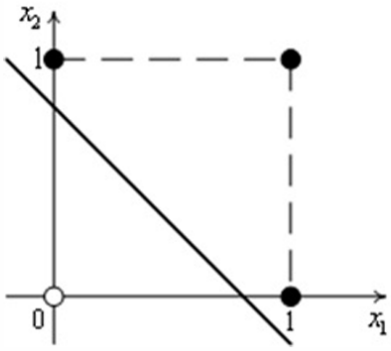


Рис.. Две полуплоскости

Унификация записи:

$$w_1x_1 + w_2x_2 + \ldots + w_nx_n \geqslant T;$$

$$-1 \cdot T + w_1x_1 + w_2x_2 + \ldots + w_nx_n = \sum_{i=0}^{n} w_ix_i = \langle w, x \rangle \geqslant 0;$$

$$x_0 = 1; w_0 = -T$$

Задача – линейно разделимая, если существуют веса w_i, т.ч.

$$\langle w, x \rangle \geqslant 0 \quad \forall x \in C_1;$$

$$\langle w, x \rangle < 0 \quad \forall x \in C_2$$

Для линейно разделимых задач применим алгоритм обучения.

Алгоритм (обучение перцептрона):

1. Задание шага обучения $\eta > 0$, k – количество строк в таблице.

2. Инициализация весов w_i небольшими случайными значениями.
   Ошибка $E := 0$.
   Номер строки таблицы: $m := 1$.
3. Начало обучения.
$$x^* := x_m, y^* := y_m$$
   Выход перцептрона:
$$y = y(x^*).$$
4. Обновление весов:
$$w := w - \eta(y^* - y)x^*.$$
5. Накопление ошибки:
$$E := E + \|y^* - y\|^2.$$
6. Если $m < k$, то $m := m + 1$ и переход на шаг 4.2.
7. Если $E = 0$, то прекращение вычислений (решение найдено), иначе $E := 0$, $m := 1$ и переход на шаг 4.2.

Существуют линейно неразделимые задачи
Проблема «исключающего или» (XOR-problem) (Минский и Пейперт):

$$y = f(x_1, x_2) = x_1 \oplus x_2$$

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

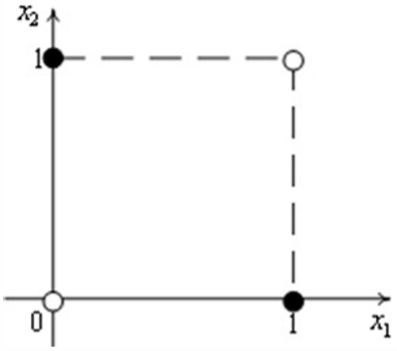


Рис.. Проблема «исключающего ИЛИ»

Нейронные сети прямого распространения (НСПР) I

Входной слой
Скрытые (промежуточные) слои
Выходной слой

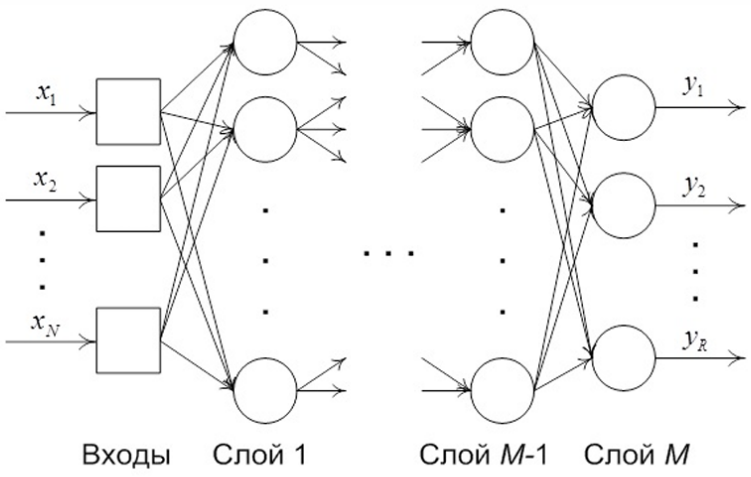


Рис.. Структура НСПР

Выход q-го нейрона m-го слоя:

$$y^{(m,q)} = \sigma^{(m,q)}\left(net^{(m,q)}\right) = \sigma^{(m,q)}\left(\sum_{i=0}^{n} w_i^{(m,q)} y^{(m-1,i)}\right), \qquad (2)$$

где $y^{(m-1,0)} = 1$, $m = 1, \ldots, M$, – единичные входы нейронов; $y^{(0,i)} = x_i$, $i = 1, \ldots, n$, – входы НСПР; $\sigma^{(m,q)}$ – ФА q-го нейрона m-го слоя.
Выход нейронной сети:

$$y(x; w) = y^{(M)} = W^{(M)}\Phi\left(W^{(M-1)}\Phi\left(\ldots\Phi\left(W^{(1)}x\right)\ldots\right)\right)$$

суперпозиционная линейно-нелинейная по параметрам структура.
Пример.

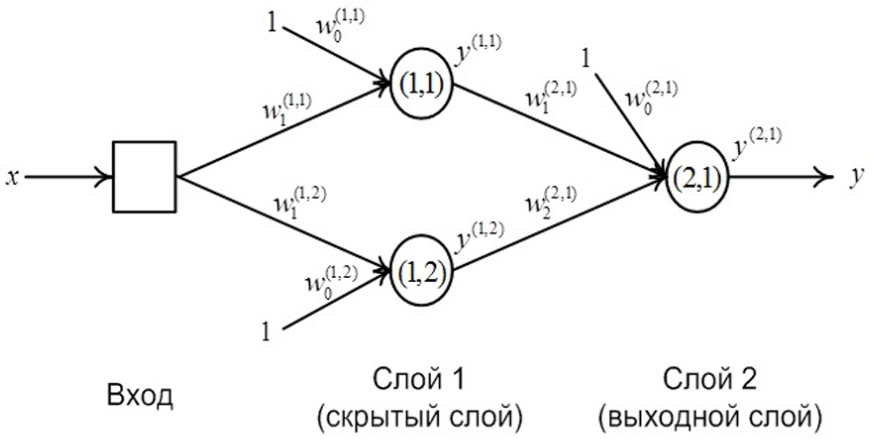


Рис.. Пример НСПР

Простейшая НСПР:

$$y = w_0 + \sum_{i=1}^{n} w_i x_i$$

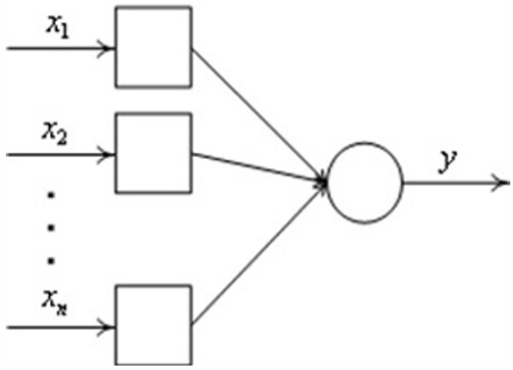


Рис.. Простейшая НСПР

Двухслойная (стандартная) НСПР (полутораслойный предиктор):

$$y = \sum_{i=1}^{q} w_i \sigma \left(\sum_{j=1}^{n} w_{ij} x_j \right)$$

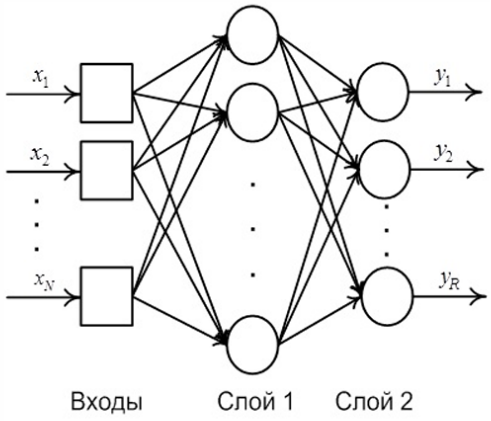


Рис.. Стандартная НСПР

Теорема (Фунахаши) об универсальных аппроксимационных способностях НСПР.

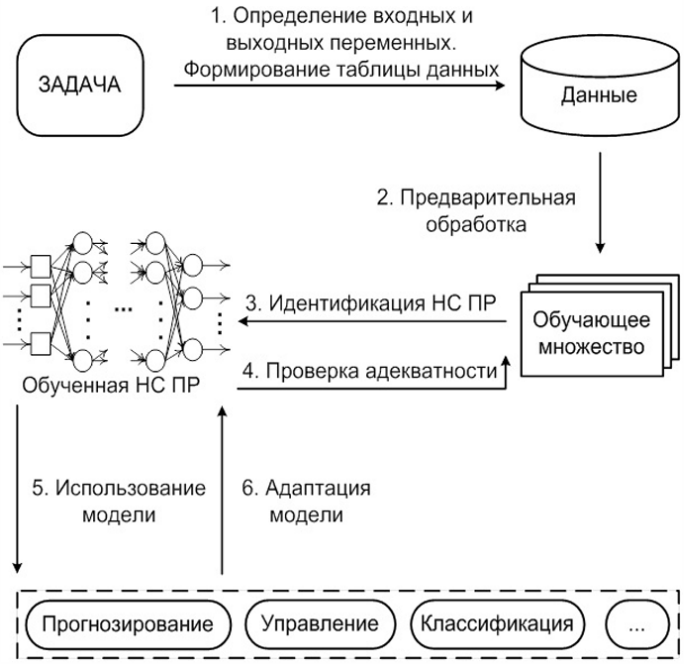
Пусть $f(x_1, x_2, \ldots, x_n)$ – непрерывная функция, определенная на компактном множестве, и $\varepsilon > 0$ – точность аппроксимации. Существует натуральное число q и набор действительных чисел w_{ij}, w_i, что функция

$$y = \sum_{i=1}^{q} w_i \sigma \left(\sum_{j=1}^{n} w_{ij} x_j \right)$$

где σ – непостоянная ограниченная монотонно возрастающая непрерывная функция (например, сигмоидная логистическая), приближает исходную функцию с погрешностью, не превышающей ε на всей области определения, т.е.

$$\sup_{(x_1, x_2, \ldots, x_n) \in \mathbb{R}^n} |f(x_1, x_2, \ldots, x_n) - y| \leqslant \varepsilon.$$

Методика применения НСПР.

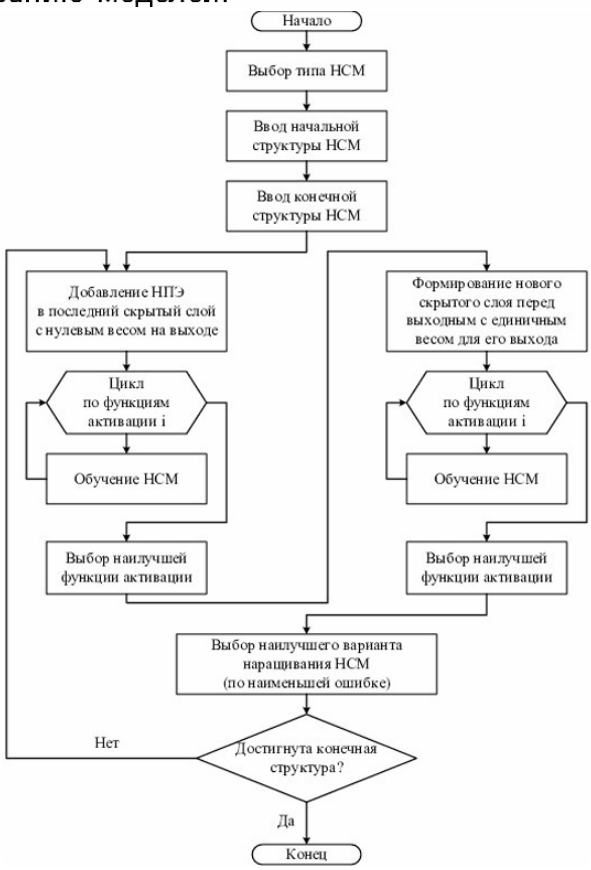


Этап 1. Информативное множество входных x и выходных y величин выбирается, исходя из поставленной задачи.
Этап 2. Преобразование данных, заданных в качественном, к числовому виду.
Восстановление отсутствующих данных в обучающем множестве (восстановление пробелов).
Удаление неестественных, ошибочных, сильно искаженных данных, не отражающих реального поведения объекта.
Переход от абсолютных значений к относительным.
Нормировка значений переменных в диапазон $[0;1]$ или $[-1;1]$.
Этап 3. Идентификация (построение) нейросетевой модели - центральный этап.
Этап 4. Основной подход: выделение из исходной таблицы вход-выходных данных тестового множества (около 20%). Остальная часть – обучающее множество. Тестовое множество состоит из данных, не использующихся в процессе обучения. Более адекватна модель, которая показывает меньшую ошибку на тестовом множестве.
Этап 5. Прогнозирование - получение выходных величин для заданных входных значений.
Управление – формирование управляющих сигналов с целью получения желаемых выходов управляемой системы.
Классификация – соотнесение входного вектора, описывающего объект, к определенному классу объектов.
Этап 6. При использовании модели с течением времени встает необходимость учета новой информации о предметной области, рассматриваемой в задаче, например, связанная с поступлением новой вход-выходной информации.
Возможны три стратегии выбора структуры НСПР:

- генерация набора НСПР, не зависимых друг от друга, и дальнейший выбор лучшей из них;
- контрастирование НСПР (редукция, network pruning), заключающееся в оценке значимости нейронов в текущей НСПР и удалении наименее значимых весов или нейронов;

- конструктивный подход (constructive, network growing), заключающийся в последовательном наращивании количества слоев и/или нейронов в скрытых слоях, начиная с сети минимальной структуры.

Подход к конструированию моделей.



ВЫВОДЫ (ОСНОВНЫЕ МОМЕНТЫ ЛЕКЦИИ)

1. Искусственный нейрон – аналог биологического нейрона.
2. Преобразование нейрона осуществляется в два этапа: вычисляется уровень активности, затем применяется функция активации.
3. Нейронные сети прямого распространения (НС ПР) обладают характерной суперпозиционной структурой.
4. Основа успешного применения НС ПР – их универсальная аппроксимационная способность.
5. Центральный этап применения НС ПР – обучение (настройка весов сети фиксированной структуры).

Машинное обучение
Лекция 9
Сверточные и глубокие нейронные сети

Сверточные нейронные сети (СНС) I

Идея – из биологии

Hubel, Wiesel, 1962: Зрительная кора имеет небольшие участки клеток, которые чувствительны к конкретным областям поля зрения. Некоторые нейроны активируются, когда воспринимают вертикальные границы, другие – когда горизонтальные или диагональные. Человек смотрит на собаку: классифицирует, если видит 4 лапы, характерный силуэт, уши и т.д.

Распознавание изображений

Компьютер выполняет классификацию изображений путем поиска низкоуровневых объектов: краев, кривых, собирая их в более высокоуровневые объекты. Это идея реализована в **сверточных нейронных сетях** (СНС) – **Convolutional Neural Networks** (CNN).

История

Архитектура СНС предложена Яном Лекуном (LeCun) в 1988 году.

Входы

Цветное изображение в формате JPG размера 480×480 в палитре RGB: массив $480 \times 480 \times 3$ чисел из диапазона 0..255.

Выходы

Вероятности того, что изображение относится к определенному классу.

Идея СНС

Переход (**не отказываются полностью**) от полносвязных слоев к **сверточным (convolution)** слоям и слоям **объединения, подвыборки (pooling, subsampling)**.
Комбинирование (чередование) слоев.
Структура – однонаправленная, принципиально многослойная (относится к группе **глубоких НС, deep NN**). Функции активации нейронов – по выбору разработчика.

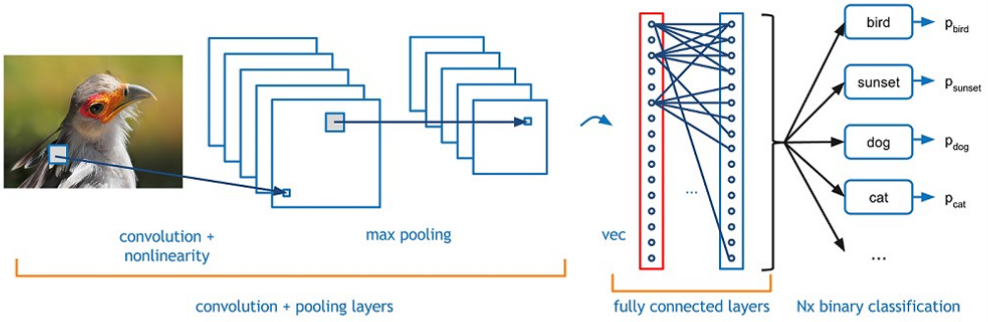


Рис.. Вид структуры СНС для задачи распознавания образов

Сверточный слой I

Пример

Цветное изображение размера в виде массива $32 \times 32 \times 3$ чисел из диапазона 0..255.

Сверточный слой

Использование фильтров, например, 5×5, заданной глубины (в нашем случае 3). Каждый фильтр – **матрица весов** (параметров), **ядро свертки**. Вычисляется поэлементные умножения ($5 \times 5 \times 3 = 75$), которые потом суммируются.
Пусть W – матрица фильтра размера $n \times n$, $X^{(l)}$ – матрица входных значений $n \times n$ для компоненты $l = R, G, B$, начиная с верхнего угла. Результат применения фильтра – скаляр s:

$$s = \sum_{l=\{R,G,B\}} \sum_{i=1}^{n} \sum_{j=1}^{n} w_{ij} X_{ij}^{(l)}.$$

Может быть: $W = W^{(l)}$.

Карта активации (карта объектов, activation map, feature map) –

результат перемещения фильтра 5×5 по всему входному изображению.

$$32 \times 32 \times 3 \to 28 \times 28 \times 1.$$

Матрицу весов передвигают обычно не на полный шаг (размер матрицы), а на небольшое расстояние: на один или два нейрона (пикселя) вместо пяти, чтобы не «перешагнуть» искомый признак.
Замечание: сравните с уровнем активации нейрона *net*.

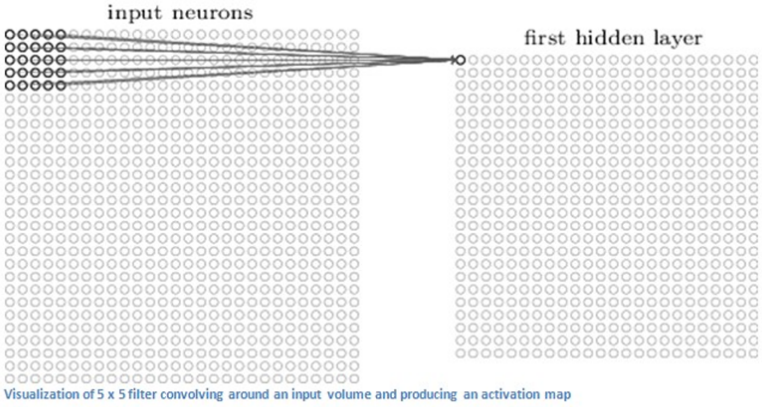


Рис.. Применение фильтра для получения карты активации

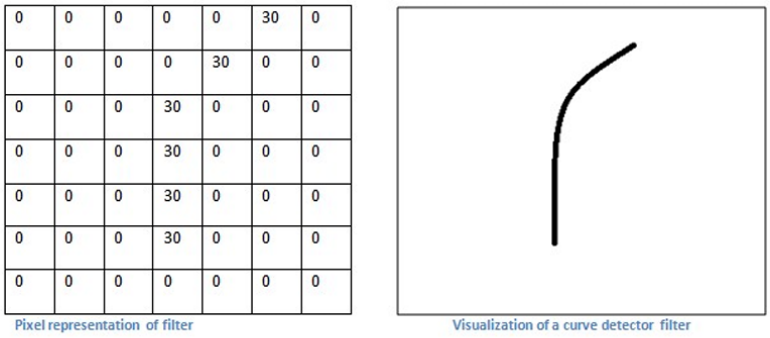


Рис.. Пример фильтра (детектора) кривой

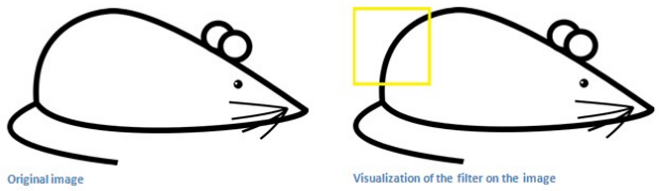


Рис.. Фильтр кривой для изображения мышки

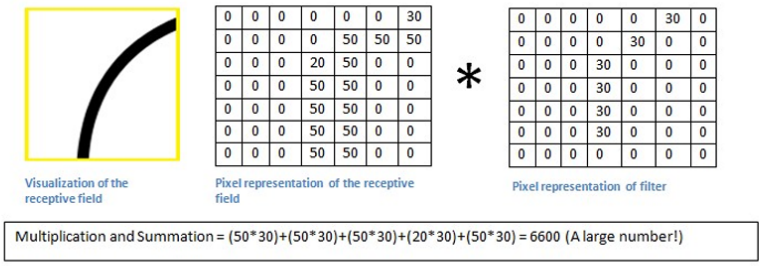


Рис.. Фильтр кривой для участка 1

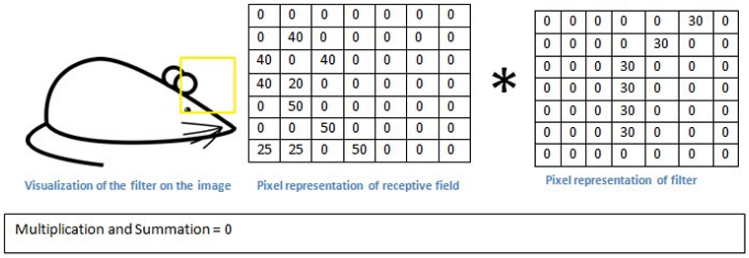


Рис.. Фильтр кривой для участка 2

Результаты применения фильтров:

- участок 1: $s = 6600$;
- участок 2: $s = 0$.

Разные фильтры

могут применяться к одному входу. Если **два** фильтра, то:

$$32 \times 32 \times 3 \to 28 \times 28 \times 2.$$

Чем больше фильтров, тем больше глубина карты свойств, тем больше информации об исходной картинке.
Возможно применение фильтров для определенного цвета (красного и т.д.).

Распространение сигналов:

выход первого – вход для 2-го слоя.
Применение 2-го сверточного слоя – свойства более высокого уровня: полукольца (комбинация прямой границы с изгибом) или квадраты (сочетание нескольких прямых ребер).

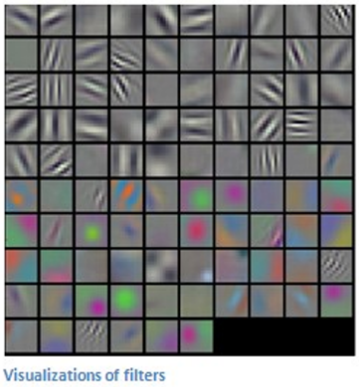


Рис.. Визуализация применения фильтров

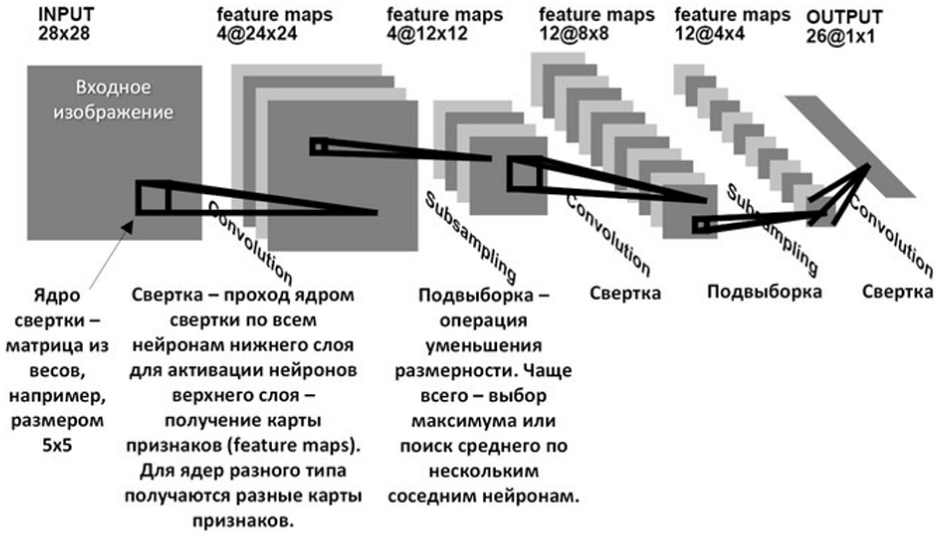


Рис.. Последовательность сверточных слоев

Иерархия признаков –

СНС «самонастраивается», «организует иерархию» абстрактных признаков (последовательности карт признаков), фильтруя маловажные детали и выделяя существенное.
Это **метафорический** характер – признаки трудны для интерпретации, суть признаков не пытаются понять.

Другие слои СНС I

Слой объединения (подвыборки) –

уменьшение размерности сформированных карт признаков.
Обычно: max или среднее по группе соседних нейронов.
Это нелинейное уплотнение карты признаков – группа пикселей (обычно 2×2) уплотняется до одного пикселя, проходя нелинейное преобразование.
Используются непересекающиеся прямоугольники или квадраты.

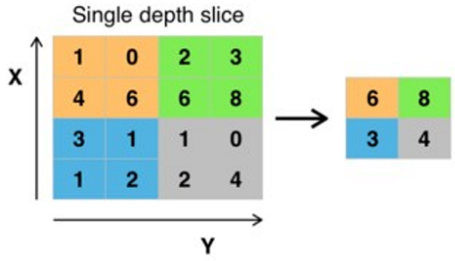


Рис.. Пример объединения на основе max

Чередование слоев –

слой объединения обычно вставляется после слоя свертки перед слоем следующей свертки.

Слой активации –

применение нелинейных функций активации (ФА) (сгмоидных, гиперболического тангенса и т.д.).
Замечание: обычно выделяются отдельными слоями.
Популярная ФА – **ReLU, Rectified Linear Unit** (с 2000-х гг.):

$$f(x) = \max(0, x).$$

Полносвязный слой (full-connected) –

вычисление выхода – вероятности определения класса $p(C_j), j = 1, \ldots, k$.

Пример – распознавание цифр

$k = 10$ для 10 цифр $(0, 1, \ldots, 9)$.
Если выход $y = [0 \quad 0,1 \quad 0,1 \quad 0,75 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0,05]$, то:

- цифра 1: $p = 0,1$;
- цифра 2: $p = 0,1$;
- цифра 3: $p = 0,75$;
- цифра 9: $p = 0,05$.

Обучение –

метод обратного распространения ошибки, стохастические методы.

Глубокие нейронные сети и глубокое обучение I

Глубокие НС (deep NN) –

НС с количеством **скрытых слоев** $\geqslant 2$.

Глубокое обучение (deep learning) –

совокупность методов машинного обучения для обучения глубоких (многослойных) НС.

СНС и глубокие НС:

СНС – тип глубокой НС.
Deep – скорее маркетинговый термин.

Архитектуры глубоких НС:

- многослойные сети прямого распространения (*многослойные перцептроны);
- сверточные нейронные сети;
- рекуррентные нейронные сети (RNN) (неглубокие RNN можно рассматривать как часть глубокого обучения, т.к. разворачиваются во времени).

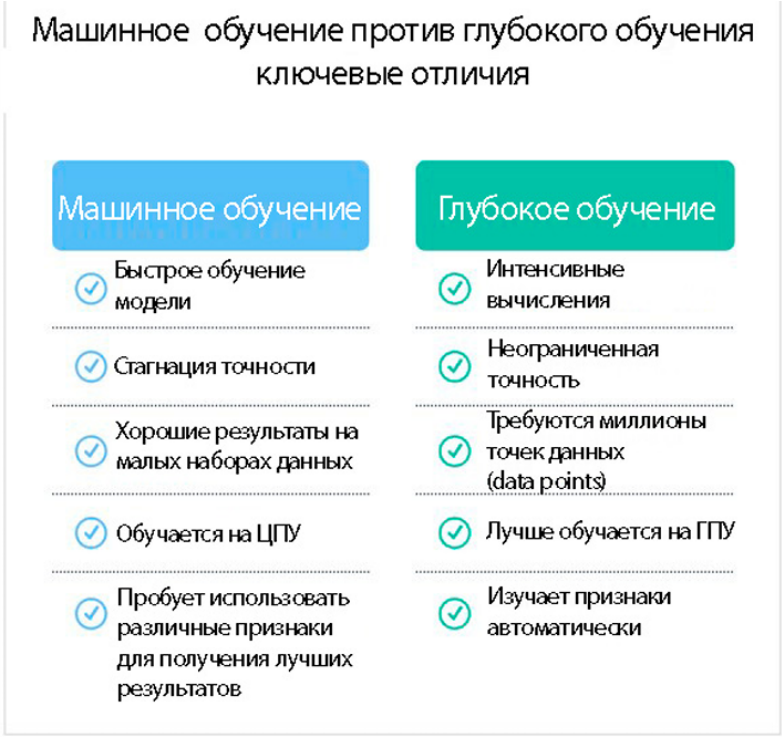


Рис.. Машинное VS глубокое обучение

ВЫВОДЫ (ОСНОВНЫЕ МОМЕНТЫ ЛЕКЦИИ)

1. СНС – архитектуры НС для распознавания образов.
2. Идея СНС – в биологии, в характере работы нейронов.
3. Архитектуры СНС – большое количество чередующихся слоев.
4. Основной слой СНС – сверточный.
5. Идея применения фильтров и слоев объединения – снижение размерности.
6. СНС – тип глубоких НС.
7. Глубокие НС – НС с количеством скрытых слоев $\geqslant 2$.
8. Глубокое обучение – разновидность методов МО для глубоких НС.

Машинное обучение
Лекция 10
Обучение нейронных сетей

Обучение нейронных сетей как задача оптимизации I

Входная информация – обучающее множество:

$$\left[\begin{array}{cccc|cccc} \tilde{x}_{11} & \tilde{x}_{12} & \dots & \tilde{x}_{1n} & \tilde{y}_{11} & \tilde{y}_{12} & \dots & \tilde{y}_{1r} \\ \tilde{x}_{21} & \tilde{x}_{22} & \dots & \tilde{x}_{2n} & \tilde{y}_{21} & \tilde{y}_{22} & \dots & \tilde{y}_{2r} \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ \tilde{x}_{k1} & \tilde{x}_{k2} & \dots & \tilde{x}_{kn} & \tilde{y}_{k1} & \tilde{y}_{k2} & \dots & \tilde{y}_{kr} \end{array}\right]$$

Здесь k – количество примеров, n и r – количество входов и выходов соответственно.

Краткая запись:

$$\langle \tilde{X}, \tilde{Y} \rangle = \{\tilde{x}_i, \tilde{y}_i\}, i = 1, \dots, k,$$

где $\tilde{x}_i \in \mathbb{R}^n$ – входной вектор из i-го примера, $\tilde{y}_i \in \mathbb{R}^r$ – вектор соответствующих указаний учителя.

Строки – **примеры**. Выходы – **указания учителя**. Далее считаем: $r = 1$.

Отклонение выхода модели сети y_i на i-м примере и соответствующего указания учителя $\tilde{y}_i$ при текущем входном векторе $x_i \in \mathbb{R}^n$ (**невязка, residual**):

$$\varepsilon_i(w) = y_i(w) - \tilde{y}_i.$$

Вектор невязок:

$$\varepsilon = \varepsilon(w) = y(w) - \tilde{y}.$$

Пусть известные значения вектора параметров $w \in \mathbb{R}^s$. Мгновенный функционал качества для примера i:

$$Q_i = Q_i(w) = Q(\varepsilon_i(w)).$$

Степень соответствия модели данным обучающего множества: интегральный функционал качества обучения:

$$Q = Q(w) = \sum_{i=1}^{k} Q_i(w)$$

Цель обучения:

$$w^* = \arg \min_{w \in \mathbb{R}^s} Q(w).$$

Обучение – задача параметрической идентификации – **оптимизационная задача**.
Q – обычно норма вектора невязок ε:

$$Q = \|\varepsilon\|.$$

Далее: 2-норма

Оптимизация (минимизация) функции одной переменной:

$$y = f(x) \to \min$$

Необходимое условие оптимума (НУО):

$$y' = f'(x) = 0$$

Достаточное условие оптимума (ДУО):

$$y'' = f''(x) > 0(\min)$$

$$y'' = f''(x) < 0(\max)$$

Пример

$$y = x^2$$

НУО:

$$y' = 2x = 0 \Rightarrow x^* = 0$$

ДУО:

$$y'' = 2 > 0 \Rightarrow x^* = 0 \Rightarrow \min$$

Оптимизация (минимизация) функции нескольких переменных

$$y = f(x_1, x_2, \ldots, x_n)$$

Необходимое условие оптимума (НУО):

$$\nabla f(x) = grad \quad f(x) = \begin{bmatrix} f'_{x_1} \\ f'_{x_2} \\ \vdots \\ f'_{x_n} \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix} = 0$$

Достаточное условие оптимума (ДУО):

$$\nabla^2 f(x) = Hessian \quad f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & & & \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

- положительно определен – т. min;
- отрицательно определен – т. max.

Пример

$$y = x_1^2 + 3x_2^2$$

НУО:

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 \\ 6x_2 \end{bmatrix} = 0$$

$$x_1^* = 0, \quad x_2^* = 0$$

ДУО:

$$\nabla^2 f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} \end{bmatrix} |_{x_1=0, \quad x_2=0} = \begin{bmatrix} 2 & 0 \\ 0 & 6 \end{bmatrix}$$

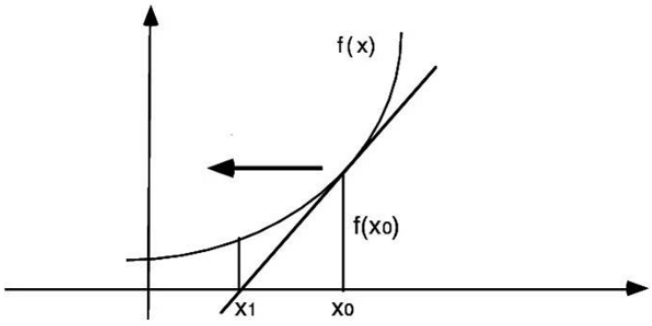
Матрица положительно определена (по критерию Сильвестра: $2 > 0$, $2 * 6 - 0 = 12 > 0$) – т. min

Если для функции одной переменной

$$f'(x) = g(x) = 0$$

не решается аналитически, применяются численные методы решения нелинейных уравнений (метод Ньютона и др.):

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}, \quad k = 1, 2, \ldots$$



Это типичный итерационный метод, идея которого – переход к x_{k+1}:

$$f(x_{k+1}) < f(x_k), \quad k = 1, 2, \ldots$$

Метод наискорейшего спуска (производная – направление возрастания функции):

$$x_{k+1} = x_k - \eta f'(x_k), \quad \eta > 0,$$

где η – шаг оптимизации – достаточно мал.
В частности:

$$\eta = \frac{1}{f''(x_k)}, \quad f''(x_k) > 0$$

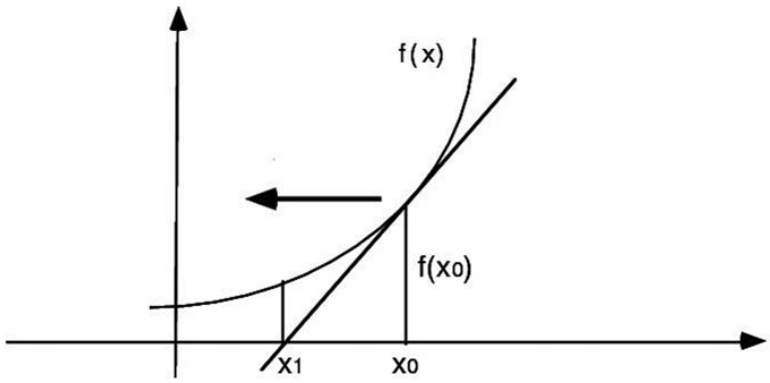


Рис.. Метод наискорейшего спуска

Функция нескольких переменных: направление возрастания функции – градиент.
Метод наискорейшего спуска:

$$x_{k+1} = x_k - \eta \nabla f(x_k), \quad \eta > 0,$$

Критерий останова:

$$\|f(x_{k+1}) - f(x_k)\| < \varepsilon,$$

где $\varepsilon > 0$ – малое число – точность.

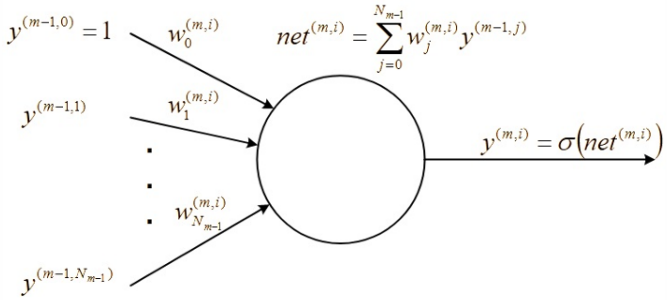
Процедура обратного распространения ошибки I

Эффективный способ нахождения градиента – процедура **обратного распространения ошибки** (ОРО) (**error backpropagation**).

Вследствие аддитивности производной:

$$\nabla_w Q(w) = \nabla_w \left(\sum_{i=1}^{k} Q_i(w) \right) = \sum_{i=1}^{k} \nabla_w Q_i(w)$$

достаточно вывести формулу вычисления градиента для мгновенного функционала качества.
i-й нейрон m-го слоя:



$$\nabla_w Q(w) = \begin{bmatrix} \vdots \\ \frac{\partial Q_i(w)}{\partial w_j^{(m,i)}} \\ \vdots \end{bmatrix}$$

Производная суперпозиции функций:

$$[f(g(x))]'_x = f'_g(g(x)) \cdot g'(x)$$

$$[f(g_1(x), g_2(x), \ldots, g_n(x))]'_x = \sum_{i=1}^{n} \frac{\partial f}{\partial g_i(x)} \cdot g'_i(x)$$

Обозначим $Q_i(w)$ через $Q(w)$.

$$\frac{\partial Q(w)}{\partial w_j^{(m,i)}} = \frac{\partial Q(w)}{\partial y^{(m,i)}} \cdot \frac{\partial y^{(m,i)}}{\partial net^{(m,i)}} \cdot \frac{\partial net^{(m,i)}}{\partial w_j^{(m,i)}}, \tag{1}$$

где последние два множителя определяются легко:

$$\frac{\partial y^{(m,i)}}{\partial net^{(m,i)}} = \sigma'_{net}(net^{(m,i)}), \tag{2}$$

$$\frac{\partial net^{(m,i)}}{\partial w_j^{(m,i)}} = y^{(m-1,j)}. \tag{3}$$

Для нахождения первого множителя формулы обозначим его как

$$s^{(m,i)} = \frac{\partial Q(w)}{\partial y^{(m,i)}}.$$

Нахождение $s^{(m,i)}$ опирается на рекуррентную процедуру, которая получается из формулы производной сложной функции нескольких переменных:

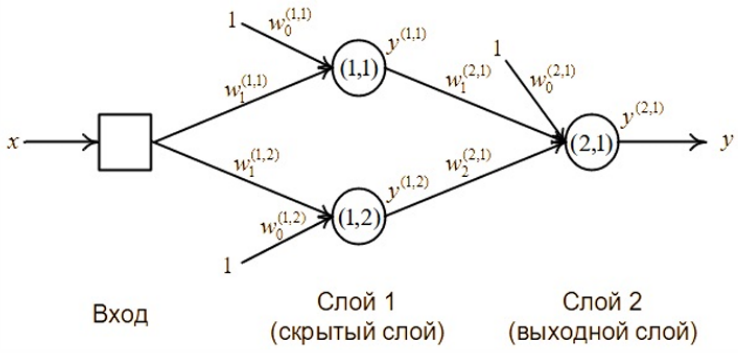
$$s^{(m,i)} = \sum_{j=1}^{N_{m+1}} \frac{\partial Q}{\partial y^{(m+1,j)}} \frac{\partial y^{(m+1,j)}}{\partial y^{(m,i)}} = \sum_{j=1}^{N_{m+1}} s^{(m+1,j)} \sigma'_{net}(net^{(m+1,j)}) w_i^{(m+1,j)}. \tag{4}$$

Для вычислений по формуле (4) необходимо начальное условие:

$$s^{(M,1)} = \frac{\partial Q(w)}{\partial y^{(M,1)}} = \varepsilon(w) = y(w) - \tilde{y}, \tag{5}$$

где $\tilde{y}$ – указание учителя текущего примера. Ошибка работы сети $\varepsilon(w)$ словно распространяется в направлении, обратном функционированию сети, что и обусловило такое название алгоритма нахождения градиента.

Пример



$$w = \begin{bmatrix} w_0^{(1,1)} \\ w_1^{(1,1)} \\ w_0^{(1,2)} \\ w_1^{(1,2)} \\ w_0^{(2,1)} \\ w_1^{(2,1)} \\ w_2^{(2,1)} \end{bmatrix}$$

$$y = y^{(2,1)} = w_0^{(2,1)} + w_1^{(2,1)} y^{(1,1)} + w_0^{(2,1)} y^{(1,1)}$$

Для последнего слоя:

$$s^{(2,1)} = \frac{\partial Q}{\partial y^{(2,1)}} = y(w) - \tilde{y}$$

Для скрытого слоя:

$$s^{(1,1)} = s^{(2,1)} \frac{\partial y^{(2,1)}}{\partial y^{(1,1)}} = s^{(2,1)} w_1^{(2,1)}$$

$$s^{(1,2)} = s^{(2,1)} \frac{\partial y^{(2,1)}}{\partial y^{(1,2)}} = s^{(2,1)} w_2^{(2,1)}$$

Далее:

$$\frac{\partial Q(w)}{\partial w_0^{(1,1)}} = s^{(1,1)} \cdot \frac{\partial y^{(1,i)}}{\partial net^{(1,i)}} \cdot \frac{\partial net^{(1,i)}}{\partial w_0^{(1,i)}}$$

Результирующий градиент:

$$\nabla_w Q(w) = \begin{bmatrix} s^{(1,1)} y^{(1,1)} (1 - y^{(1,1)}) \\ s^{(1,1)} y^{(1,1)} (1 - y^{(1,1)}) x \\ s^{(1,2)} y^{(1,2)} (1 - y^{(1,2)}) \\ s^{(1,2)} y^{(1,2)} (1 - y^{(1,2)}) x \\ s^{(2,1)} y^{(2,1)} (1 - y^{(2,1)}) \\ s^{(2,1)} y^{(2,1)} (1 - y^{(2,1)}) y^{(1,1)} \\ s^{(2,1)} y^{(2,1)} (1 - y^{(2,1)}) y^{(1,2)} \end{bmatrix}$$

Поиск глобального минимума при обучении

Функционал качества, который обучается,– многоэкстремален.
Решение – добавление к ОРО **стохастичности** (на основе метода Монте-Карло):

- метод **мультистарта**:
  1. из множества X случайно или детерминировано выбирается некоторое подмножество из N точек;
  2. последовательно из каждой точки запускается алгоритм локального спуска, и из полученного множества локальных решений выбирается наилучшее;
- метод **встряски весов** (jog of weights) = метод туннелирования – каждая итерация из двух фаз:

1. фаза минимизации улучшает значение текущего рекорда;
2. фаза туннелирования находит точку из допустимого множества, отличную от последней, где найдено значение минимума. Полученная на фазе туннелирования точка рассматривается как стартовая для очередного цикла.

Глубокое обучение и его особенности I

Глубокое обучение (deep learning) –

совокупность методов машинного обучения для обучения глубоких (многослойных) НС.

Обучение –

метод обратного распространения ошибки, стохастические методы.

Проблема затухания градиента

Затухающий градиент (Vanishing Gradient): в процессе ОРО элементы градиента становится все меньше. Проблема: эти веса изменяются на слишком малые значения, обучение фактически не проводится (чем меньше номер слоя, тем меньше обучение). Причина: ограничение производной ФА (униполярная сигмоидная логистическая – [0; 0,25], биполярная сигмоидная логистическая – [0; 1]) и использование большого количества слоев. При насыщении этих ФА производная $\to 0$.

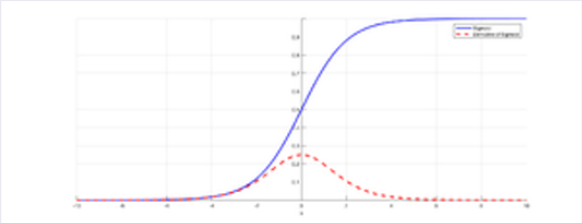


Рис.. Униполярная сигмоидная логистическая фнукция и ее производная

Пусть $w^{(k)}$ – вес некоторого нейрона на слое k, $x^{(k)}$ – некоторый вход этого нейрона.
Пусть $Q = \frac{1}{2}MSE$.

$$\frac{\partial Q}{\partial w^{(k)}} = \frac{\partial Q}{\partial y} \cdot \frac{\partial y}{\partial w^{(k)}} = (y - \tilde{y})\sigma' x^{(k)} \leqslant \frac{1}{4}(y - \tilde{y})x^{(k)}$$

$$\frac{\partial Q}{\partial w^{(k-1)}} = \frac{\partial Q}{\partial w^{(k)}} \cdot \frac{\partial w^{(k)}}{\partial w^{(k-1)}} \leqslant \left(\frac{1}{4}\right)^2 (y - \tilde{y})x^{(k)}x^{(k-1)}$$

Решение 1 – использование других ФА:

Производная ReLU (недифференцируемость при $x = 0$ игнорируется):

$$\frac{d}{dx} ReLU(x) = \begin{cases} 1, & x > 0; \\ 0, & x \leqslant 0. \end{cases}$$

Устранение недифференцируемости – Softplus (используется редко из-за усложнения):

$$\sigma(x) = \ln(1 + e^x).$$

Решение 2 – использование остаточного обучения



Рис.. Residual Learning

$$y = x + F(x)$$

Градиент выхода слоя по входу (отсутствие проблемы затухания):

$$\frac{\partial y}{\partial x} = 1 + \frac{\partial F(x)}{\partial x}$$

Решение 3 – регуляризация весов:

слишком большие значения весов увеличивают значение функционала качества.
LASSO:

$$Q + \lambda \|w\|_1$$

Гребневая (Ridge) регрессия:

$$Q + \lambda \|w\|_2$$

Здесь $\lambda > 0$ – параметр регуляризации.

Решение 4 – инициализация весов Ксавье/Глорота (Xavier/Glorot):

инициализация малыми числами (около нуля).
Пусть N_{in} – число входов нейрона, N_{out} – число выходов нейрона.

$$w_i \cong N(0, \sigma),$$

$$\sigma = \sqrt{\frac{1}{N_{in} + N_{out}}}.$$

Проблема взрывного роста градиента

Взрывающийся градиент (**Exploding Gradient**): в процессе ОРО при прохождении через слои в элементах градиента могут накапливаться большие значения. Проблема: возможно переполнение, другие веса не обновляются.
Причина: в НС с обратными связями при ВРТТ, используются одни и те же веса.

Решение – градиентная обрезка (Gradient Clipping) (масштабирование)

- Обрезка по норме (Clipping-by-Norm):

$$\nabla Q_w(clipped) = \begin{cases} \nabla Q_w & \|\nabla Q_w\| \leqslant T; \\ \nabla Q_w \cdot \frac{T}{\|\nabla Q_w\|}, & \|\nabla Q_w\| > T. \end{cases}$$

- Обрезка по значению (Clipping-by-Value):

$$\nabla Q_{wi}(clipped) = \begin{cases} \nabla Q_{wi} & |\nabla Q_{wi}| \leqslant T; \\ T, & |\nabla Q_{wi}| > T. \end{cases}$$

Здесь T –пороговое значение.

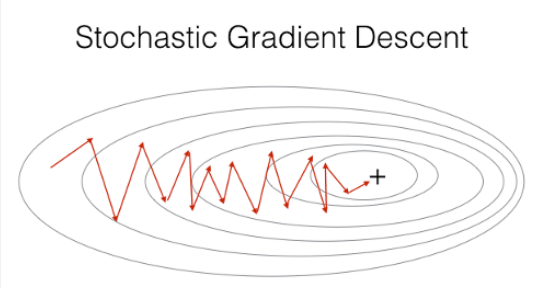
Метод стохастического градиента и mini-batch метод I

Цель – повышение эффективности обучения на обучающем множестве **огромного объема**.

Стохастический градиентный спуск (Stochastic Gradient Descent, SGD):

корректировка весов после использования одного **случайно выбранного** примера за один так обучения. Это позволяет:

- повысить скорость прохождения этапов обучения;
- избежать локальных экстремумов.



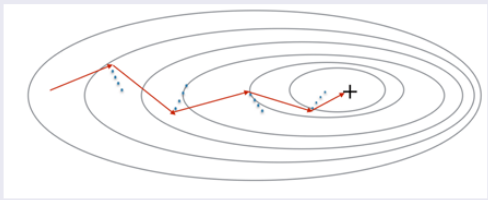
SGD с импульсом (SGD with momentum):

на каждой итерации используется скользящее среднее градиента. Цель – использование инерции в обучении. Используется приближенное скользящее среднее градиентов на предыдущих этапах:

$$g_0 = \nabla Q_{w0}; \quad g_t = \alpha g_{t-1} + \eta \nabla Q_{wt};$$

$$w_{t+1} = w_t - g(t).$$

Здесь g_t – накопленное среднее градиентов на шаге t, $\alpha \in [0, 1]$ – степень сохранения истории значений (обычно ≈ 0.9).



Пакетный градиентный спуск (Batch Gradient Descent, BatchGD):

расчет градиента функционала качества как суммы градиентов по всем примерам (сумма всех мгновенных функционалов качества). Это затратно по памяти.

Mini-batch градиентный спуск (Mini-Batch Gradient Descent):

комбинация SGD и BatchGD – случайное подмножество заданного размера из примеров обучающего множества.

ВЫВОДЫ (ОСНОВНЫЕ МОМЕНТЫ ЛЕКЦИИ)

1. Обучение НС ПР в формализуется как задача оптимизации.
2. Эффективный способ вычисления градиента функционала качества по вектору весов – процедура обратного распространения ошибки.
3. Важно учитывать многоэкстремальный характер функционала качества при обучении НС.
4. Глубокое обучение – разновидность методов МО для глубоких НС.